



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG



AI-based Anomaly Detection for Electric Vehicle Supply Equipment using LSTM Architecture

An der Fakultät für Informatik und Mathematik der
Ostbayerischen Technischen Hochschule Regensburg
im Studiengang
Wirtschaftsinformatik

eingereichte

Bachelorarbeit

zur Erlangung des akademischen Grades des
Bachelor of Science (B.Sc.)

Vorgelegt von: Yannic Hanel
Matrikelnummer: 3296954

Erstgutachter: Prof. Dr. Rudolf Hackenberg
Zweitgutachter: Prof. Dr. Sebastian Fischer

Abgabedatum: 23.08.2025

Erklärung zur Bachelorarbeit

1. Mir ist bekannt, dass dieses Exemplar der Abschlussarbeit als Prüfungsleistung in das Eigentum der Ostbayerischen Technischen Hochschule Regensburg übergeht.
2. Ich erkläre hiermit, dass ich diese Abschlussarbeit selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Regensburg, den August 17, 2025

Yannic Hanel

Abstract

AI is important ...

Contents

I	List of Abbreviations	VII
1	Introduction	1
1.1	Motivation and Relevance of Anomaly Detection in Electric Vehicle Supply Equipment (EVSE)	1
1.2	Overview of AI-based approaches	4
1.3	Research Questions	5
1.4	Structure of the Thesis	6
2	Theoretical Background and Core Concepts	7
2.1	Fundamentals of Anomaly Detection	7
2.2	Data Processing and Visualization Tools	9
2.3	Introduction to Decision Trees, Random Forest, Isolation Forest and LSTM . . .	10
2.4	Overview of LSTM (Long Short-Term Memory) Architecture	12
3	Related Work	14
3.1	Recent Advances in Cybersecurity and Anomaly Detection for EVSE	14
3.2	Performance Comparison of Deep Learning Approaches for EVSE Anomaly Detection	14
3.3	General Anomaly Detection Methods and Foundations	15
4	Data Acquisition	17
4.1	Data Sources for EVSE Anomaly Detection	17
4.2	Integration of Prometheus	18
5	Exploratory Data Analysis	20
5.1	Holistic Dataset Comparison	20
5.2	Feature-Level Comparison of CICEVSE2024 and Self-Collected Datasets	25
6	Feature Selection	32
6.1	Data Preprocessing Steps	32
6.2	Feature Selection Using Decision Trees	33
7	LSTM Architecture: Structure, Training, and Optimization	41
7.1	LSTM Architecture and Preprocessing	41
7.2	Parameter Configuration and Hyperparameter Optimization	42
8	Analysis and Evaluation	47
8.1	Experimental Setup and Evaluation Metrics	47
8.2	Results of Anomaly Detection Using the Machine Learning Model	48

9 Discussion	52
10 Conclusion/Future Work	53
Anhang	VIII
A Domändenmodell	VIII

List of Figures

1	Availability of public charging points and market share of electric cars in the EU [ACEA23].	1
2	EVSE as an access node to critical data[EntryPoints25].	2
3	Interior view of the wallbox[Han25].	3
4	An illustration of the position of machine learning (ML) and deep Learning (DL) within the area of artificial intelligence (AI)[AI-Definition]	4
5	Data distribution classified by Type 1 – Outlier Clustering. The data is adapted from the Wine dataset (Blake and Merz, 1998)[HA04].	7
6	Z-score with critical region[Per19]	8
7	Decision Tree[DecisionTreeExplain24]	11
8	setup EV Charing Station from [EDBF24]	21
9	Selected HPC and Kernel Features from [EDBF24]	22
10	Boxplot of the CICEVSE2024 Dataset Features	23
11	Boxplot of the self-collected Dataset Features	24
12	L1-icache-loads timeline (CICEVSE2024)	25
13	L1-icache-loads timeline (self-collected)	25
14	instruction timeline	26
15	instruction timeline (CICEVSE2024)	26
16	cache-misses timeline	27
17	cache-misses timeline (CICEVSE2024)	27
18	exc-taken timeline	28
19	exc-taken timeline (CICEVSE2024)	28
20	cpu-migrations timeline	29
21	cpu-migrations timeline (CICEVSE2024)	29
22	dTLB-stroe-misses timeline	30
23	dTLB-stroe-misses timeline (CICEVSE2024)	30
24	Comparison of feature selection methods for LSTM accuracy [Kot24].	34
25	Top 10 most important features selected by the Decision Tree.	35
26	Correlation matrix top Features with Decision Tree.	36
27	Correlation matrix of top features selected by the Decision Tree on the CICEVSE2024 dataset.	37
28	Correlation between <code>skb_kfree_skb</code> and <code>fib_fib_table_lookup</code> features.	38
29	Timeline behavior of the <code>node-stores</code> feature, showing distinguishable patterns during flooding attacks.	39
30	Timeline behavior of the <code>sys_calls_sys_enter_kill</code> feature, highlighting its usefulness in detecting port scan attacks.	39
31	Timeline behavior of the <code>unaligned_st_spec</code> feature, showing little variation between benign and attack data.	40

32	Balanced label distribution between the training and test datasets.	43
33	syscalls_sys_exit_recvfrom timeline	48
34	bus_access_not_shared timeline	49
35	qdisc_qdisc_dequeue timeline	49
36	l2d_cache_wb timeline	49
37	syscalls_sys_enter_sendto timeline	50
38	Confusion matrix	50
39	Falsely labeled attacks	51
40	Average attack_prob values per attack type compared to the benign baseline. Flooding attacks show significantly higher average probabilities than the benign reference, while slow and scanning attacks remain close to or even below it. . . .	51

List of Tables

1	Comparison of machine learning approaches with examples, advantages, and disadvantages. Own illustration based on [AI-Definition].	5
2	Comparison of supervised learning models with advantages and disadvantages. Own illustration based on [DecisionTreeExplain24][Bre01][LTZ08][SM19].	12
3	Strengths and Limitations of LSTM. Own illustration based on [SM19].	13
4	Overview of executed attacks	32
5	Attack intervals during data collection	33
6	Explanation of key components used in the LSTM model configuration.[Aro+21][Sae][Ram+18][KE	

I List of Abbreviations

Electric Vehicle Supply Equipment (EVSE)
Electric Vehicles (EVs)
Charging Stations (CS)
Artificial Intelligence (AI)
Isolation Forest (iForest)
Recurrent Neural Network (RNN)
Exploratory Data Analysis (EDA)
Deep Neural Network (DNN)
Application Programming Interface(s)(APIs)
Information Technology (IT)
Denial-of-Service (DoS)
Principal Component Analysis (PCA)
man-in-the-middle (MitM)

1 Introduction

1.1 Motivation and Relevance of Anomaly Detection in Electric Vehicle Supply Equipment (EVSE)

Electric Vehicle Supply Equipment (EVSE) has gained significant importance in recent years [EVSE24]. Analyzing the market sales of electric vehicles (EVs), it can be observed a remarkable growth: in 2012, only 120,000 EVs were sold, whereas by 2021, sales had surged to nearly 6.6 million. To achieve net-zero CO₂ emissions by 2050, as planned by the European Union [EU2050], the number of EVs must continue to rise substantially—requiring an 11-fold increase from current levels by 2030 alone [IEA22].

To manage the much higher demand for EVs on public roads, it is expected that the market for EVSE will grow similarly to the sales of EVs [ACEA23]. The correlation between the number of EVs and the number of EVSE is high. If we analyze the market data from the last few years, it becomes evident that the top four countries in the European Union like Germany, Netherlands, France, and Belgium with the best EVSE infrastructure are also the top four with the largest market share of EVs [ACEA23].

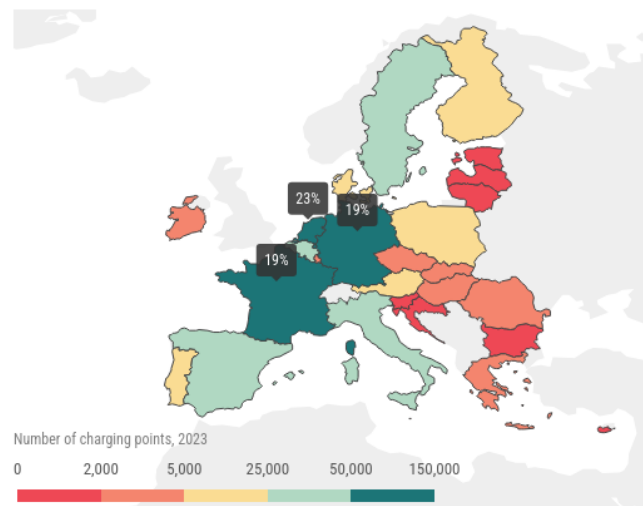


Figure 1: Availability of public charging points and market share of electric cars in the EU [ACEA23].

With the growing number of EVSE, particularly Charging Stations (CS), the interest of cybercriminals groups in attacking these systems is also increasing [Ram]. At first glance, EVSE systems may not appear to be direct targets for accessing critical data or manipulation. However, they can serve as entry points to gain access to more sensitive or private information [EntryPoints25]. The following figure illustrates how EVSE components, such as charging stations, can act as access nodes to critical or private data.

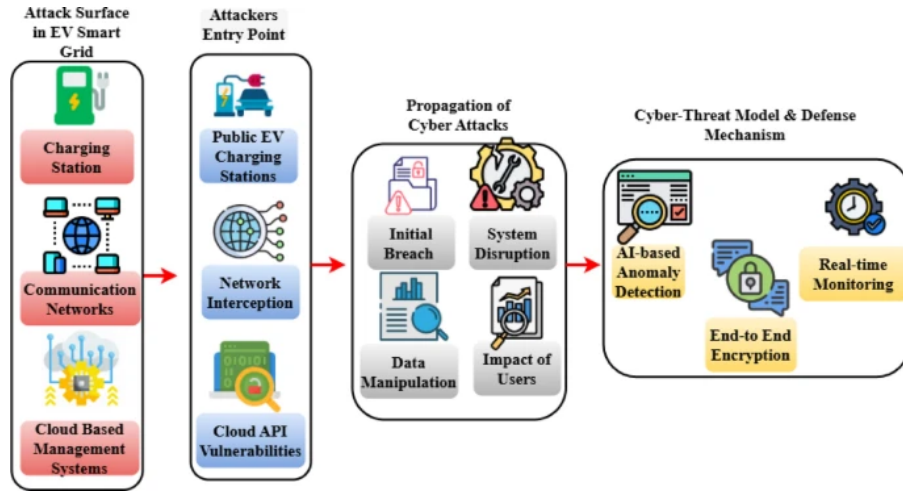


Figure 2: EVSE as an access node to critical data[EntryPoints25].

With the access to these kinds of data the critical attacks like data manipulations, data theft, disruption of service up to complete system failure are possible[EntryPoints25]. This does not only come with a high financial risk, it also affects the trust of the users in the EVSE systems and potentially slows down the adoption of EVs in general.

If we provide end users with a stable and secure EVSE infrastructure, consumer confidence in switching to electric vehicles will be significantly higher. This, in turn, ensures a consistent growth in the adoption of both EVs and EVSE systems[EVSE-Cyber25].

As seen in the first paragraphs, the rapid growth of EVs and EVSE infrastructure also requires a robust monitoring and anomaly detection system to ensure the security of the EVSE infrastructure. Only if we can ensure the security of the EVSE infrastructure can we maintain user trust in the energy transition and avoid major cybersecurity headlines in the future.

Ensuring high security standards for EVSE systems is of critical importance, yet difficult to achieve. The EVSE infrastructure constitutes a complex system that includes not only charging stations but also sophisticated backend systems and communication protocols. These components and software solutions are typically developed, published, and maintained by various vendors, which makes securing the entire system a challenging task.

Combining all the different software from vendors into a secure infrastructure is challenging enough, but the EVSE infrastructure, especially charging stations (CS), is also regularly installed in public places. Even when not in a public setting, these installations are often outdoors, where security measures are significantly lower than in buildings protected by walls, doors, and other physical barriers.

In this paper, we focus on wallboxes, which are generally smaller than CS and are typically used in private settings. They are often installed directly on house walls or in garages, and thus still form part of the overall EVSE infrastructure.

The fact that they are installed in private areas initially suggests a higher level of security.

However, since these areas are still outdoors, we must assume that there are no additional physical security measures beyond the wallbox itself. Therefore, it is crucial to develop a system that not only protects against software-based attacks, such as Denial-of-Service (DoS) or Distributed Denial-of-Service (DDoS), but also considers other potential attack vectors such as firmware manipulation and unauthorized data interception. In addition, the system must secure the hardware against physical threats like tampering, theft, or other intrusions. In our project, we use wallboxes from the [ReSiLENT] project, provided by [StoehrMobility]. The software is developed by [ChargeIQ] and already includes basic security measures to protect against external threats. This project is also carried out in cooperation with [ASVIN]. In the following section, we will take a closer look at the internal components of the wallbox.

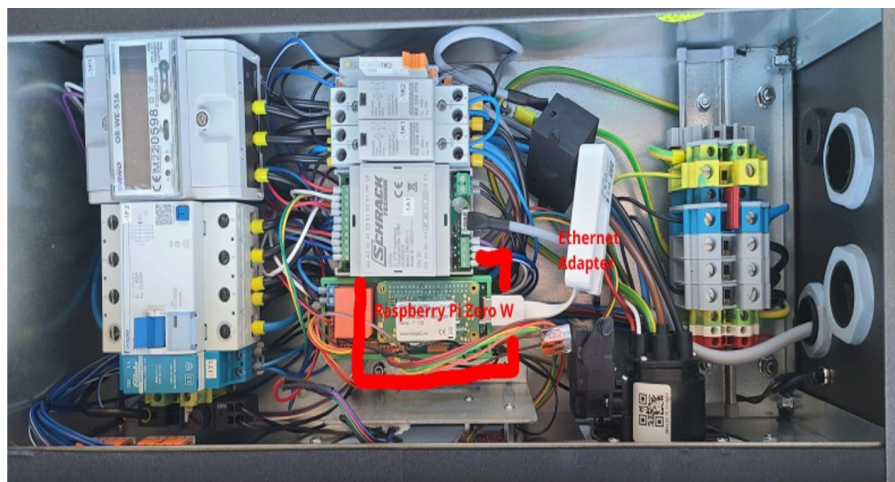


Figure 3: Interior view of the wallbox[Han25].

In this figure, the internal components of the wallbox are visible. The wallbox is equipped with a Raspberry Pi Zero W, which serves as the main controller for the planned peer-to-peer communication between the wallboxes. This device is also intended to host of the detection system developed as part of this project.

Located behind the Raspberry Pi Zero W is another Raspberry Pi, which is responsible for communication with the backend, including the transmission of user data and handling of payment processes from the Charging process. This component is secured and programmed by *ChargeIQ GmbH*[ChargeIQ].

The sensitive hardware components are enclosed in an aluminum casing designed by *Stöhr Mobility GmbH*[StoehrMobility], which provides a degree of protection against physical attacks. However, upon closer inspection of these physically secured components, it must be acknowledged that the protective measures can be relatively easily bypassed. This has already been observed and documented by our colleague Lea Laux in her work [Laux24].

As already mentioned, the wallbox also processes personal data of users, such as their user ID and payment information, in communication with the backend server. This requires special

attention in the context of the DSGVO / GDPR, as additional requirements for the protection of personal data like data minimization, storage limitation [GDPR5] and legal obligation [GDPR6] must be taken into account throughout the entire EVSE infrastructure.

Security requires significant computational resources, particularly in the domain of anomaly detection. As shown in Figure 3, the wallbox is equipped with a Raspberry Pi Zero W, which offers relatively sufficient resources for a single-board computer. However, when processing and analyzing over 1,000 hardware events to detect potential anomalies, the system quickly reaches the limits of its hardware capabilities this is just a small example of the challenges we face in securing EVSE infrastructure.

Due to the combination of digital and physical threats, the involvement of multiple vendors, and strict data protection requirements, securing Electric Vehicle Supply Equipment (EVSE) infrastructure is a complex task and brings a lot of Challenges.

1.2 Overview of AI-based approaches

Artificial intelligence is often used in different contexts, therefore we have to define what we mean by AI-based approaches and what different approaches exist. At first it is a leading technology in the current digital transformation and is crucial to move forward the fourth industrial revolution[AI-Definition].

The Goal of AI is to create machines that can perform tasks that typically require human intelligence, such as understanding natural language and recognize patterns in data. Especially detection of patterns is in this work from Relevance. We can divide the different AI-based approaches into two sub categories:

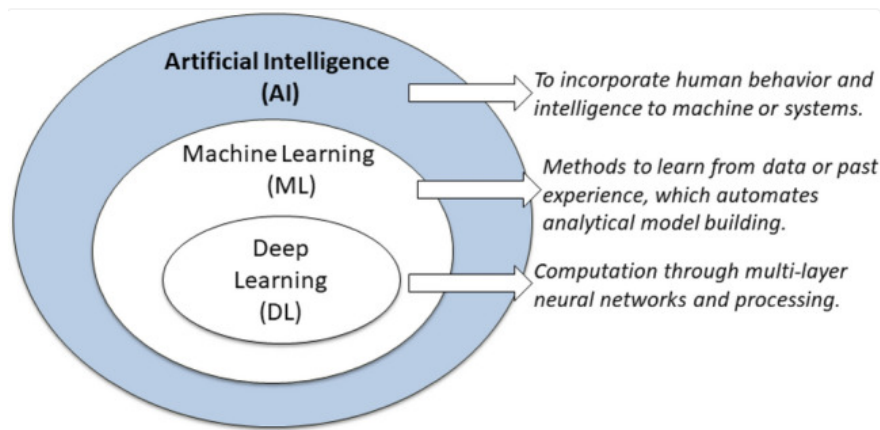


Figure 4: An illustration of the position of machine learning (ML) and deep Learning (DL) within the area of artificial intelligence (AI)[AI-Definition]

these can be further divided into three categories:

- **Supervised Learning:** This approach involves learning from labeled data to identify patterns and make predictions. It is commonly used in tasks such as email spam classification or price prediction, where the algorithm is trained on known input-output pairs.

- **Unsupervised Learning:** This data-driven method is used to uncover hidden patterns in unlabeled data. Techniques such as clustering and dimensionality reduction are applied to identify structures within the data, for example, to segment customer groups or discover business models.
- **Semi-Supervised Learning:** This approach combines aspects of both supervised and unsupervised learning by utilizing a small amount of labeled data alongside a larger pool of unlabeled data. It is particularly useful when labeling data is expensive or time-consuming, but large amounts of unlabeled data are available.

[AI-Definition]

In the following table, we categorize the different approaches alongside their corresponding machine learning architectures. This allows us to identify the advantages and disadvantages of each method and to gain a clearer understanding of which approach is most suitable for producing actionable results in real-world deployments.

Approach	Examples	Advantages	Disadvantages
Supervised Learning	Decision Trees, Random Forest, LSTM	High prediction accuracy, Clear objectives	Requires a lot of labeled data, Risk of overfitting
Unsupervised Learning	K-Means, Isolation Forest	Finds patterns without labels, Useful for unlabeled data	Hard to interpret, No clear target
Semi-Supervised Learning	Self-training	Improves performance with few labeled data	Complex to implement

Table 1: Comparison of machine learning approaches with examples, advantages, and disadvantages. Own illustration based on [AI-Definition].

After gaining an overview of the different machine learning approaches, we can now delve deeper into the field by examining various methods for anomaly detection and identifying the most effective models for detecting typical anomalies.

1.3 Research Questions

The following research questions guide this thesis on **AI-based Anomaly Detection for Electric Vehicle Supply Equipment using LSTM Architecture**:

- What types and characteristics of anomalies occur in electric vehicle supply equipment (EVSE)?

- How can relevant features be identified and selected to effectively train machine and deep learning models for anomaly detection?
- How does the performance of LSTM-based models compare to that of traditional or alternative AI-based methods in terms of detection accuracy and false positives?
- What practical challenges arise when implementing such anomaly detection systems in real-world charging infrastructure?

These research questions address both the technical and real-world aspects of developing and applying anomaly detection, providing the analytical framework for the subsequent chapters of this thesis.

1.4 Structure of the Thesis

This thesis is divided into ten chapters, each building upon the previous to explore the use of AI-based methods—specifically LSTM architectures—for anomaly detection in Electric Vehicle Supply Equipment (EVSE).

Chapter 1 introduces the motivation and relevance of the topic, outlines the role of anomaly detection in the context of EV infrastructure, and formulates the central research questions. Additionally, it provides an overview of prior AI-based approaches and concludes with this structural outline. Chapter 2 summarizes the key theoretical foundations necessary for the thesis, including the basics of anomaly detection, a short introduction to tree-based models such as decision trees and random forests, and a focused explanation of the LSTM architecture.

Chapter 3 presents related work, covering research on security aspects and anomaly detection mechanisms specific to EVSE, as well as a broader review of AI-based detection methods applied in similar domains. Chapter 4 describes the dataset creation and acquisition process, detailing both public datasets and custom setups involving Prometheus monitoring tools. Chapter 5 follows with an exploratory data analysis (EDA), intended to uncover key patterns and differences between the public and self-generated datasets.

In Chapter 6, the process of data preprocessing and feature selection is discussed, including the identification of relevant features and the use of feature importance methods such as decision trees. Chapter 7 focuses on the LSTM model itself, describing its architecture, the training process, and the chosen strategies for hyperparameter tuning.

The results of the approach are then evaluated in Chapter 8, which outlines the experimental setup, evaluation metrics, and key results of the applied anomaly detection system. Chapter 9 provides a detailed discussion of the findings in relation to the research questions, highlighting limitations and potential practical applications. Finally, Chapter 10 concludes the thesis with a summary of the work and a reflection on future directions.

This structure ensures a logical and methodical development of the topic, from theoretical foundations to implementation, evaluation, and critical interpretation.

2 Theoretical Background and Core Concepts

2.1 Fundamentals of Anomaly Detection

In the previous section, we took a closer look at what Artificial Intelligence is and how useful it can be in the field of EVSEs. In this section, we delve deeper into the area of Anomaly Detection and its fundamental concepts.

Anomaly Detection is the process of analyzing data patterns to identify outliers — in other words, data points that deviate significantly from the norm. This technique can be applied in a wide range of domains, including credit card fraud detection, insurance and healthcare monitoring, intrusion detection in cyber-security, fault detection in safety-critical systems, military surveillance, and also in the field of EVSEs.[BAN09]

Outliers are data points that differ significantly from the majority of the data. In the paper by Hodge and Austin, they define outliers as “data points that are inconsistent with the rest of the data.” The paper also includes some informative graphics that illustrate the different types of outliers and how they can be detected.[HA04]

Type 1 is the most common form of outlier, where data points are generally clustered around

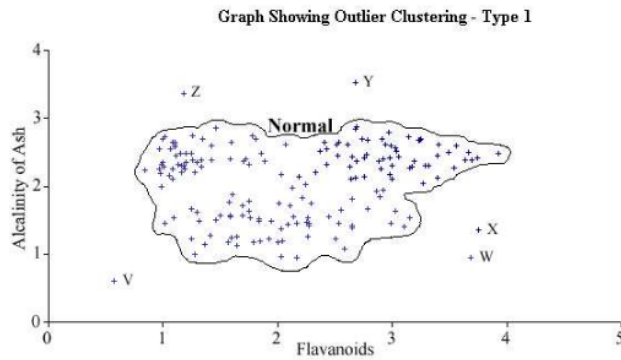


Figure 5: Data distribution classified by Type 1 – Outlier Clustering. The data is adapted from the Wine dataset (Blake and Merz, 1998)[HA04].

a central value, but a few points differ significantly from the rest. This can be seen in the figure above, where most data is centered, yet a few outliers deviate clearly.

Such outliers can be detected using a simple z-score algorithm, where the mean μ and the standard deviation σ of the data are calculated. The z-score is then computed using the formula: $Z = \frac{X - \mu}{\sigma}$. Estimated z-scores greater than 3 or less than -3 can be considered anomalies. These anomalies should be given increased attention during data analysis. Additionally, anomalies can be contextualized by parameters such as time or location to improve their identification.[Per19]

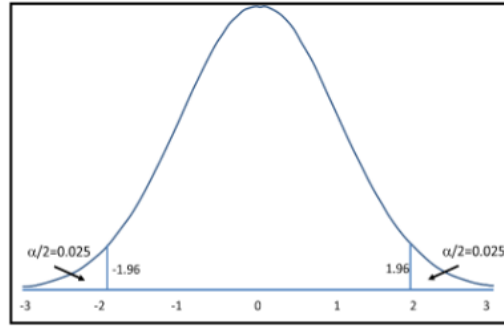


Figure 6: Z-score with critical region[Per19]

But before you can calculate the z-score you need an extensive data collection and preprocessing. This is also the first step in the Anomaly Detection process. The data must be cleaned, normalized, and transformed to ensure that it is suitable for analysis. This may involve removing duplicates, handling missing values, and scaling the data to a common range.

After the data is preprocessed, feature engineering is performed to extract relevant features from the data. Feature engineering involves selecting and transforming the data; this can sometimes also create new features that can improve the performance of the anomaly detection model. Also, data reduction is a relevant step in this process, where the data is reduced to a smaller set of features that are most relevant for the anomaly detection task.[Mah22]

If you are done with the data collection and preprocessing, you have to select a suitable model for your case of anomaly detection. But before we analyse the different models, let's take a look at the typical challenges of anomaly detection.

Like mentioned before to detect anomalies we have to analyze data patterns to identify outliers. But to determine the correct border between normal and abnormal data can be challenging and depends extremely on the context of the data.

The EVSE infrastructure is changing rapidly, not just with new technologies; the charging stations change with every new charging process and communication with other CS in the network. This makes it difficult to define a static outlier threshold. Instead, it's extremely important to adapt the model to the current state of the EVSE network and the charging process to generate dynamic detection in all situations in the dynamic EVSE environment.[BAN09]

Before the z-score can be calculated, extensive data collection and preprocessing are required. This forms the first step in the Anomaly Detection process. The data must be cleaned, normalized, and transformed to ensure it is suitable for analysis. This may involve removing duplicates, handling missing values, and scaling the data to a common range.

After preprocessing, feature engineering is performed to extract the most relevant characteristics from the dataset. Feature engineering involves selecting and transforming the data — and sometimes even creating new features — to improve the performance of the anomaly detection model. Additionally, data reduction is a crucial part of this process. It focuses on reducing the dataset to a smaller subset of features that are most relevant to the anomaly detection task.[Mah22] This step is of high relevance for the dataset used in this work, with further details

provided in the *Data Acquisition and Preprocessing* chapter.

Once data collection and preprocessing are completed, the next step is to select a suitable model for the specific anomaly detection use case. But before we analyze the different models, it is important to examine the typical challenges associated with anomaly detection.

As previously mentioned, detecting anomalies requires analyzing data patterns to identify outliers. But determining the line between normal and abnormal behavior is challenging and highly dependent on context.

The EVSE infrastructure is also evolving rapidly — not only due to new technologies, but also because charging stations behave differently during each new charging process and through communication with other charging stations in the network. This variability makes it difficult to define a static outlier threshold. Therefore, it is essential to adapt the anomaly detection model to the current state of the EVSE network and the ongoing charging processes. This enables dynamic detection capabilities that remain effective under all conditions in a highly dynamic EVSE environment.[BAN09]

2.2 Data Processing and Visualization Tools

This adaptive approach of Anomaly detection ensures that detection mechanisms remain robust and reliable, even as EVSE infrastructures and their operational conditions continue to evolve. This section introduces the data processing and visualization tools used to implement such adaptable anomaly detection strategies.

To convert the data into more useful formats, I used **Pandas**, a fast, powerful, and easy-to-use open-source data analysis tool for Python. It provides expressive data structures such as `DataFrame` and `Series` for working with structured and time series data.

I primarily used Pandas to convert my data, which was originally stored in `.csv` files, into `.parquet` files. Additionally, I used it to manipulate the dataframes by dropping cells, adding information such as "Attack" or "Benign" tags to almost 7000 rows. This preprocessing step was essential to clean the data before applying machine learning algorithms.

Some other useful key features of Pandas include:

- Tools for reading and writing data between various formats
- Flexible reshaping and merging of data
- Powerful grouping, aggregation, and transformation operations

[The25]

To create diagrams that help me better understand the data, I used a popular plotting library in Python called **Matplotlib**. Specifically, I used the `matplotlib.pyplot` module, which is a state-based interface to the Matplotlib library and provides a MATLAB-like way of creating static, animated, and interactive visualizations in Python.

Matplotlib is well-suited for quickly prototyping visualizations thanks to its simple and readable

syntax. This allowed me to create a wide range of plots, including, Line Plots, Scatter Plots, Bar Charts, Histograms and Heatmaps.

These visualizations enabled me to better understand the data structure, identify patterns, and prepare the dataset for further processing.

I also used the Seaborn library it is like the other 2 libraries also an open-source Python library. It is especially known for making statistical graphics built on top of the matplotlib lib. Seaborn provides a high-level interface for plotting informative and attractive statistical plots. It is also closely integrated with Pandas data structure what allows easy visualization of whole datasets and enables - Plotting of complex relationships - automatic aggregation and statistical estimation (Means, confidence intervals, etc) - Thematic styling and enhanced color palettes. Therefore it was for me a perfect addition to the pandas and Matplotlib libraries. [Wt25]

In addition to Matplotlib and Pandas, I also used the open-source Python library **Seaborn**. Seaborn is especially known for creating statistical graphics and is built on top of the Matplotlib library. It provides a high-level interface for drawing informative and attractive statistical plots. Seaborn is closely integrated with Pandas data structures, which makes it easy to visualize entire datasets. This integration enabled me to create complex visualizations with minimal code. Key features of Seaborn include:

- Plotting of complex relationships between multiple variables
- Automatic aggregation and statistical estimation (e.g., means, confidence intervals)
- Thematic styling and enhanced color palettes for better aesthetics

For these reasons, Seaborn was a perfect addition to the Pandas and Matplotlib libraries in my data analysis workflow. [Wt25]

2.3 Introduction to Decision Trees, Random Forest, Isolation Forest and LSTM

In this section, we will provide a brief introduction to several machine learning models who are categorized as supervised learning models. These models are also the most relevant for our project.

The **Decision tree** classifiers are the most well known and one of the powerful models in various fields. It works by creating a tree-like structure with Nodes and Branches and every Node is an Attribute or feature and the Branch can referent the characteristic of the Attribute[DecisionTreeExplain24].

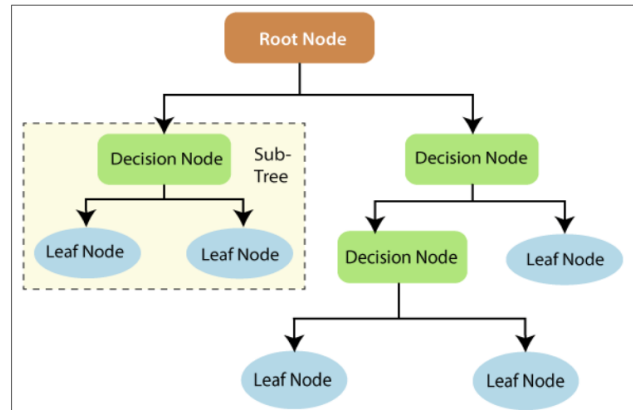


Figure 7: Decision Tree[DecisionTreeExplain24]

The decision rule is not a black box and can be easily interpreted. It compares the value of the feature with a threshold and then decides which branch to follow. It is not only used in Machine Learning, but is also common in fields like image processing and pattern recognition, due to its simple analysis and high accuracy. Despite these advantages, it has a high risk of overfitting, especially with small datasets.[DecisionTreeExplain24]

The **Random Forest** is an ensemble method that combines multiple decision trees that work independently of each other. Each decision tree is created using a random vector to improve variance and reduce overfitting.

The final decision is made by majority vote of all the trees together. This method is more robust against overfitting and can handle large datasets with high dimensionality. Two key vectors influence the performance of the Random Forest: the correlation between the trees and the strength of the individual trees.[Bre01]

The **Isolation Forest** is an innovative approach for anomaly detection. While conventional methods typically create a model of "normal" behavior and classify deviations from it as anomalies, the iForest follows the principle of directly isolating anomalies.[LTZ08]

The explanation for this is that anomalies are characterized by their rarity and by being significantly different in their attributes. Therefore, they are easier to separate from the majority. This model brings advantages such as robustness with regard to dimensionality and irrelevant features. On the other hand, *swamping* is one of the disadvantages, where normal instances that are too close to anomalies can prevent anomalies from being isolated and thus make them harder to detect.[LTZ08]

A **Long Short-Term Memory (LSTM)** is a special type of Recurrent Neural Network (RNN) designed to model sequential data and long-term dependencies. Especially the long-term dependencies have high priority in this project, where the focus is on detecting anomalies over longer time periods. LSTMs use memory cells and gating mechanisms to store information over long time steps reliably. Despite their many advantages, they come with many different parameters, making them difficult to tune. Additionally, they represent a black-box model, meaning it is hard to understand why the model made a certain decision.[SM19]

Machine Learning Model	Advantages	Disadvantages
Decision Tree	Easy to understand and interpret	High possibility of overfitting
Random Forest	Less overfitting than a single decision tree; handles high-dimensional data	Not well-suited for small datasets
Isolation Forest	Robust to high-dimensional and irrelevant features	Swamping effect in presence of nearby normal instances
Long Short-Term Memory	Effective at modeling long-term dependencies	Black-box nature and complex parameter tuning

Table 2: Comparison of supervised learning models with advantages and disadvantages. Own illustration based on [DecisionTreeExplain24][Bre01][LTZ08][SM19].

2.4 Overview of LSTM (Long Short-Term Memory) Architecture

In this chapter, I want to provide a detailed overview of the LSTM architecture, as it is the most frequently used and complex model in my work.

Long Short-Term Memory, or LSTM for short, was developed in 1997 by Sepp Hochreiter and Jürgen Schmidhuber as a solution to the vanishing gradient problem that occurs in traditional Recurrent Neural Networks (RNNs) [SM19]. It allows the network to learn dependencies over much longer time sequences, which is particularly important in my datasets, where attacks can be insidious and not immediately obvious.

This addresses one of the core limitations of traditional RNNs, which struggle to learn long-term dependencies due to the vanishing gradient problem. When training these networks, the error signals either explode—causing oscillating weights and unstable learning—or vanish, preventing the network from learning dependencies over extended time periods. Standard RNNs can typically bridge about 5 to 10 time steps, whereas LSTMs are capable of learning dependencies over 1,000 or more time steps [SM19], LSTMs process inputs sequentially, with a constant runtime of $\mathcal{O}(1)$ per time step.

But how does this work, and how is it possible to achieve this with constant time complexity? The LSTM architecture consists of six different internal states that are processed in a specific sequence to achieve this goal.

- **Forget Gate:** The forget gate calculates a dynamic value between 0 and 1 that scales the previous cell state. A value of 0 means "forget everything" and a value of 1 means "keep everything". This allows the network to selectively forget information that is no longer relevant.
- **Input Gate:** The Input gate decide how much of the new information should be added

to the new cell state at the current time step. It is a kind of protective mechanism that tries to prevent irrelevant inputs from being added to the cell state.

- **Candidate Values:** The candidate values are the new information that get in form of squashed values that get written into the memory cell.
- **Cell State Update:** The cell state is updated by simultaneously scaling down the existing state via the forget gate and adding in the gated candidate values. This allows the network to maintain a stable memory while also incorporating new information.
- **Output Gate:** The output gate determines which parts of the cell state should be really becomes the output of the LSTM and protecting in this way the network from sharing unnecessary information too early.
- **Hidden State:** The hidden state is the final output of the LSTM for the current time step and what the output gate allows to be shared with the next time step.

[SM19]

Machine Learning Model	Advantages	Disadvantages
Long Short-Term Memory	Excellent long-term memory capabilities	Fixed network topology – cannot dynamically adjust memory capacity
	Selective information processing through gates	Complexity – more parameters than simple RNNs
	Stable learning without gradient explosion/vanishing	Modularization challenges – unclear how to optimally structure networks

Table 3: Strengths and Limitations of LSTM. Own illustration based on [SM19].

3 Related Work

3.1 Recent Advances in Cybersecurity and Anomaly Detection for EVSE

In the emerging field of EVSE, numerous studies focus either on electric vehicle supply equipment itself or on anomaly detection methods. However, the specific combination of anomaly detection applied to EVSE is not yet widely explored. Therefore, the paper [EDBF24], titled *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset*, is highly relevant to this research. The authors present a dataset used for anomaly detection that includes similar features such as hardware events, power consumption, and other HCP-related data. The paper focuses on various attack scenarios and attempts to detect them using several machine learning models.

Another relevant contribution in this field is the paper [EVSE24], titled *Regression Based Anomaly Detection in Electric Vehicle State of Charge Fluctuations Through Analysis of EVCI Data* which not only highlights the importance of anomaly detection in EVSE, but also provides a comprehensive overview of a realistic charging station scenario. The setup includes four different charging boards, six charging ports, and a battery energy storage system (BESS) handling various types of EVs. The study monitors the state-of-charge (SoC) values of EVs, compares them with expected values, and detects outliers that may indicate anomalies. These detections are performed using machine learning methods within a dynamic environment.

In the context of EVSE as part of critical infrastructure, recent research highlights its role as a potential attack vector within complex smart grid environments. Studies report threats such as denial-of-service (DoS), man-in-the-middle (MitM), spoofing, and data manipulation. Modern approaches increasingly integrate anomaly detection methods using machine learning to identify and mitigate such attacks at an early stage. While not all technical implementation details are relevant here, the findings emphasize the need for advanced security mechanisms to ensure the integrity and availability of EVSE systems [EntryPoints25].

3.2 Performance Comparison of Deep Learning Approaches for EVSE Anomaly Detection

In the following, it is important to summarize which machine learning and deep learning algorithms have shown promising performance in the field of anomaly detection for EVSE. Since the data involves time series and hardware-related measurements, the paper [Ali20], titled *Deep Learning-based Intrusion Detection System for Electric Vehicle Charging Station*, provides a valuable overview of suitable deep learning models in this context.

The authors compare a Deep Neural Network (DNN) with a Long Short-Term Memory (LSTM) model and conclude that the LSTM architecture achieves high accuracy—over 99%—after only

a few training epochs. This supports the later decision in this thesis to select the LSTM model for better performance and anomaly detection in EVSE environments.

But the LSTM is only as strong as the significance of its features. Therefore, a good feature selection for the LSTM algorithm is essential to achieve an effective deep learning approach for anomaly detection. The paper by Kottilingal (2024) [Kot24] offers a systematic comparison of different feature selection methods in combination with an LSTM network for intrusion detection. The authors evaluate both the detection accuracy and the efficiency of the LSTM model when applying various feature selection strategies. The central finding is that reduced feature sets—such as those selected using tree-based methods—can achieve almost the same detection accuracy as larger sets, while significantly lowering model complexity and computational requirements. This aspect is particularly relevant for resource-constrained environments such as embedded EVSE systems.

In the context of EVSE, this means that implementing a practical and high-performance anomaly detection system requires not only choosing a suitable deep learning model (e.g., LSTM) but also effectively reducing the input dimensionality through feature selection. As already shown in [EDBF24], even with EVSE-specific data, selecting a smaller but relevant subset of hardware and kernel events can maintain detection accuracy while significantly reducing system load. The work by Kottilingal supports this approach with experimental results and provides practical recommendations for combining deep learning architectures with feature selection, making it a valuable reference for objective method comparisons in smart charging anomaly detection.

3.3 General Anomaly Detection Methods and Foundations

In the field of anomaly detection, the paper *Anomaly Detection: A Survey* [BAN09] serves as a central methodological foundation. It provides a systematic classification of approaches, including cluster-based and statistical methods, and explains their respective strengths, weaknesses, and typical use cases. Furthermore, it differentiates between various types of anomalies, such as point anomalies and contextual anomalies, and discusses the most suitable detection strategies for each category. This makes it an indispensable reference for modern anomaly detection procedures.

Building on this established methodological framework, a solid basis is created upon which more recent approaches, particularly deep learning models and methods tailored specifically to EVSE, can be developed. This also includes the LSTM-based anomaly detection presented in this work. The paper therefore highlights both the breadth and depth of anomaly detection research and ensures that specialized techniques are embedded within a well-founded conceptual context.

Another relevant contribution in the broader context of cybersecurity is presented by Rahman et al. (2024), titled *ChronoCTI: Temporal Pattern Extraction from Cyber Threat Intelligence*

Reports [Rah+25]. The authors propose an automated pipeline that leverages modern machine learning techniques and natural language processing to extract recurring, time-based sequences of malicious activities from unstructured reports. In contrast to traditional approaches that primarily focus on static indicators such as IP addresses or malware signatures, this work emphasizes the analysis of how an attack evolves over time.

The findings demonstrate that incorporating the temporal progression of malicious activity into detection processes can significantly enhance threat identification. For EVSE cybersecurity, this perspective is particularly valuable, as certain attacks on charging infrastructure may unfold gradually, with patterns becoming apparent only when their temporal dimension is considered. Such an approach aligns closely with the strengths of LSTM-based anomaly detection, which is well-suited to capturing dependencies over time and can thereby improve the detection of subtle, long-running attacks.

4 Data Acquisition

4.1 Data Sources for EVSE Anomaly Detection

In line with the approach described in the paper [EDBF24] *Enhancing EV Charging Station Security Using A Multi-dimensional Dataset : CICEVSE2024*, I recorded Hardware Counter Performance (HCP) kernel events and tracepoint events to classify anomalies in Electric Vehicle Supply Equipment (EVSE) data. For efficient monitoring, I also utilized the **perf** tool, a robust Linux performance analysis utility capable of capturing a variety of performance data, including hardware events, software events, and tracepoints [con25]. This tool offers flexible and effective monitoring of system performance and is instrumental in diagnosing issues efficiently.

To perform the data acquisition on a Raspberry Pi Zero W, I used the **perf.exporter**[Hod19] to record up to 900 Tracepoint-events, which were selected based on the CICEVSE2024 dataset. Initially, I developed a Python script to capture kernel and HCP events. However, when expanding to about 110 events, my developed script's runtime increased to approximately 9 seconds, which proved insufficient for real-time analysis. Consequently, I investigated the internal workings of the **perf** tool to understand how it manages specialized events and optimizes recording.

As detailed in *Linux perf event Features and Overhead*[Wea15], **perf** supports multiplexing, allowing the measurement of more events than there are available hardware counters—albeit at the cost of accuracy loss and increased system overhead. Recent updates can result in resource usage exceeding 20%, compared to approximately 2% in older tool versions. To minimize these inaccuracies, I grouped events per CPU, exploiting up to 8 available counter registers and limiting each group to a maximum of 6 events.

For the monitoring interval, I followed recommendations from the literature[Lim+14], which addresses multiplexing and accuracy trade-offs. The suggested minimum monitoring interval is around 10 ms for reliable results. Balancing the number of counter registers and striving for good measurement accuracy, I configured groups of 6 hardware events per group, with a monitoring interval of 0.1 seconds. This configuration yielded data with sufficient accuracy to detect anomalies in the EVSE dataset.

To further optimize efficiency and accuracy, I transitioned to developing the data acquisition script in Go. The Go programming language offers low-level hardware access and, with available libraries, integrates more seamlessly with the **perf** tool compared to Python. Additionally, the script exports all monitored hardware events from **perf** into a Prometheus database, enabling centralized and convenient access to all collected data. This setup streamlines subsequent analysis, visualization, and anomaly detection activities.

Instead of including the full script, its functionality can be summarized as follows. The script

collects system performance metrics using the Linux **perf** tool and filters the relevant events specified in a YAML configuration file. It then formats these metrics in a way that is compatible with Prometheus and makes them available through an HTTP **/metrics** endpoint. This allows the performance data to be monitored continuously and integrated into observability platforms in a structured manner.

4.2 Integration of Prometheus

In the research on which database would be suitable for centralized data storage and analysis, **Prometheus** was a key focus. Not only is it an open-source project [20125], but it is also well known in the industry as a reliable monitoring solution for complex systems. It has a large community, along with comprehensive documentation and strong support, which are essential for maintaining a stable IT environment.

Another reason for choosing **Prometheus** is its seamless integration with Grafana for visualization, as well as its powerful PromQL query language. Additionally, **Prometheus** can store metrics in a time series database, which is particularly important for our detection systems [BMM24].

Prometheus consists of several components that are required for its operation. The core of the system is the **Prometheus** server, which collects and stores metrics and provides access to them through the PromQL query language.

Client libraries are directly integrated into the application code and expose a client-server interface. The **Prometheus** server periodically fetches data from them — by default every 15 seconds, although this interval is configurable [SM24].

As explained in the previous subsection, **Prometheus** collects data via metrics exporters (see code snippet 9). After the data is collected and stored centralized in the **Prometheus** time series database, we developed a Python script that allows exporting the data within a defined time range and with specified time steps.

This export functionality is essential for enabling further analysis, including exploratory data analysis, using external tools such as Python or statistical software. The following paragraph presents the Python script developed to export **Prometheus** data into a structured CSV format, enabling its direct use in subsequent analytical workflows.

The Python script is designed to systematically retrieve and organize time-series metrics from a Prometheus monitoring server, facilitating subsequent analysis and interpretation. The workflow begins by defining a specific time range for the data of interest and partitioning this interval into smaller, manageable chunks, which helps prevent overwhelming the server with excessively large queries and ensures efficient data retrieval. For each feature specified in an external configuration file, the script constructs appropriate Prometheus queries, attempting first to obtain the standard metric and, if unavailable, a performance-prefixed variant. The retrieved data are then parsed and organized according to their respective timestamps, forming a structured

representation of the system's behavior over time. Finally, the script outputs these metrics in a wide-format CSV file, where each row corresponds to a timestamp and each column corresponds to a specific metric. This approach not only allows for straightforward integration with downstream data analysis or visualization workflows but also ensures reproducibility, scalability, and clarity in the presentation of high-dimensional monitoring data.

5 Exploratory Data Analysis

5.1 Holistic Dataset Comparison

So far, we have provided an overview of the relevance and challenges involved in securing EVSE environments. We discussed the fundamental knowledge required before developing an anomaly detection system, gained insight into related work in this area, and explored how it can inform our own approach.

We have also examined how data can be acquired from EVSE environments and how **Prometheus** can be used to collect and store this data.

All of these sections and subsections are essential for understanding the context of our work and for showing how it is possible to develop an anomaly detection system for EVSE environments based on the analysis of hardware events and the collected data.

In this section, we focus on the Exploratory Data Analysis (EDA) of the collected data. EDA is a crucial step in understanding the data, identifying patterns, and preparing for further analysis or modeling. We use Python and its data analysis libraries within a Jupyter Notebook environment to visualize and explore the collected data. The Python libraries utilized for data processing and visualization are explained in more detail in Chapter 2.2 (Fundamentals), providing a foundational understanding of the tools applied in this analysis. Establishing this methodological basis is crucial, as the quality of the results in the following chapters and significantly influences how well anomaly detection using machine learning models will perform and how quickly they will learn. Therefore, we need to answer the following questions:

- Is there any noise in the data, and how significant is it?
- Can we observe meaningful distributions in the features over time and in relation to attacks?
- How strong are the correlations between the different features?
- Can we identify and define the most important features?

Answering these questions through Exploratory Data Analysis (EDA) provides a foundation for effective feature selection and model performance [Goo] furthermore we will compare all results with the dataset CICEVSE2024 to guaranty to have a meaningful result and can transfer the results to other charging station setups.

We start with analyzing the framework conditions of both datasets, the public dataset CICEVSE2024 and the self-collected dataset.

The paper *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset* [EDBF24] and my work pursue the same goal: detecting anomalies through the analysis of HCP and kernel events. Therefore, it is essential to compare the underlying datasets on which the machine learning algorithms are trained. Such a comparison enables validation of the respective approaches and helps determine whether the results can be generalized.

We tried to get as close as possible to the environment of the paper *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset*

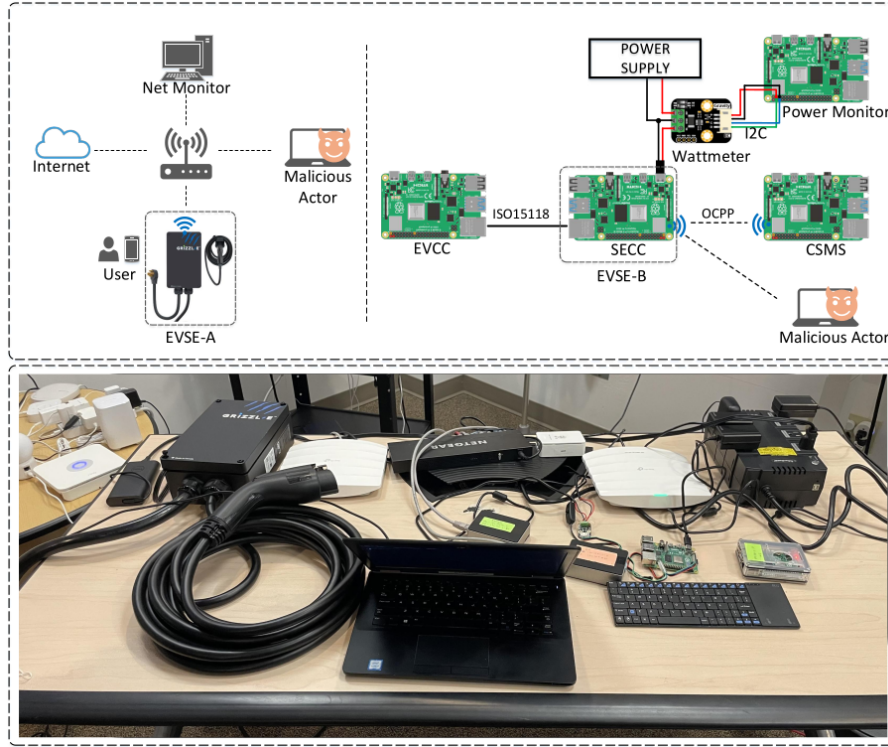


Figure 8: setup EV Charing Station from [EDBF24]

We also used a Raspberry Pi to represent the EVSE-B and employed a wattmeter to monitor power consumption. The combination of HCP events, kernel events, and power consumption measurements will be discussed in future work. As mentioned in Chapter **Data Sources for EVSE Anomaly Detection**, we also used the `perf` tool to monitor the various events.

Therefore, we are able to observe approximately 900 events in our dataset, which is comparable to the reference dataset. The complete observation period spans from 2025-07-22T18:00:00 to 2025-07-23T12:52:10, with a sampling interval of 5 seconds, identical to the compared dataset [EDBF24]. This results in approximately 8,000 data rows and 905 columns. Importantly, no missing (NaN) values are present, and all measurements were collected using the `perf` tool, producing logically consistent outputs as described in the chapter *Data Sources for EVSE Anomaly Detection*.

To ensure comparability, features distributed across multiple CPU cores (e.g., `cpu=0,1,2,3`) were aggregated using appropriate `perf` flags. This not only prevents data loss but also reduces

the number of distinct features, thereby aligning the dataset structure more closely with the reference dataset CICEVSE2024.

To establish a clear starting point for the later feature analysis, we first refer to the selected features presented in the paper *Enhancing EV Charging Station Security Using A Multi-dimensional Dataset: CICEVSE2024* [EDBF24]. These selected features provide a foundation for comparison and help guide our own selection process.

Event	Description	Backdoor	Crypto	Dos	Recon
instructions	Number of instructions executed	214.19	943.54	316.65	130.69
cache-misses	Number of cache misses	39.92	397.06	112.83	64.70
exc.taken	Exception taken	86.82	61.42	169.62	44.55
cpu-migrations	CPU Migrations	842.43	-100.00	1425.70	110.05
dTLB-store-misses	Data TLB - Write Misses	39.56	1068.57	101.48	72.97
l1d.cache_wr	Level 1 data cache access - Write	143.87	121.56	321.24	113.14
L1-icache-loads	Level 1 instruction cache access - Read	163.10	538.63	283.92	107.18
l2d.cache_rd	Level 2 data cache - Read	51.47	422.95	151.50	70.75
mem_access_rd	Data memory access - read	118.19	387.56	253.17	91.42
mem_access_wr	Data memory access - write	142.94	119.96	322.17	114.86
kmem.kfree	Kernel memory freeing event	149.06	67.66	2087.07	187.25
net_dev_xmit	Network device transmission event	27.42	33.79	172.88	59.64
qdisc.dequeue	Dequeue event	31.47	42.66	261.16	88.42
raw_syscalls_sys_enter	System call entry (raw) event	107.66	79.70	2.35	44.13
irq_softirq_raise	Software interrupt - Raised	90.29	65.72	1573.65	109.45
sched_migrate_task	Task migration event in the scheduler	670.49	-100.00	1091.74	78.18
sched_switch	Task switch event in the scheduler	42.10	39.22	332.82	24.59
syscalls_sys_enter_close	System call entry for close syscall	421.15	273.72	15.58	231.63
syscalls_sys_enter_read	System call entry for read syscall	44.32	30.19	37.09	56.21
syscalls_sys_enter_write	System call entry for write syscall	119.30	35.73	57.34	76.92

Figure 9: Selected HPC and Kernel Features from [EDBF24]

The CICEVSE2024 dataset itself consists of the same 905 event types and comprises approximately 8,500 rows. Timestamps are recorded as floating-point values representing the time of each measurement. Like our dataset, it was collected using the `perf` tool, ensuring high precision and consistency across all recorded events [EDBF24]. Out of the entire dataset, only four rows contain NaN values across all features.

At first glance, one might assume a high level of noise when considering the Z-Score. However, in this case, the values do not represent random noise or errors but instead correspond to peak values during attacks. Thus, the actual noise level in both datasets is very low, which can also be observed later in the direct feature comparison. Such rows with high peaks can therefore be considered meaningful signals rather than noise.

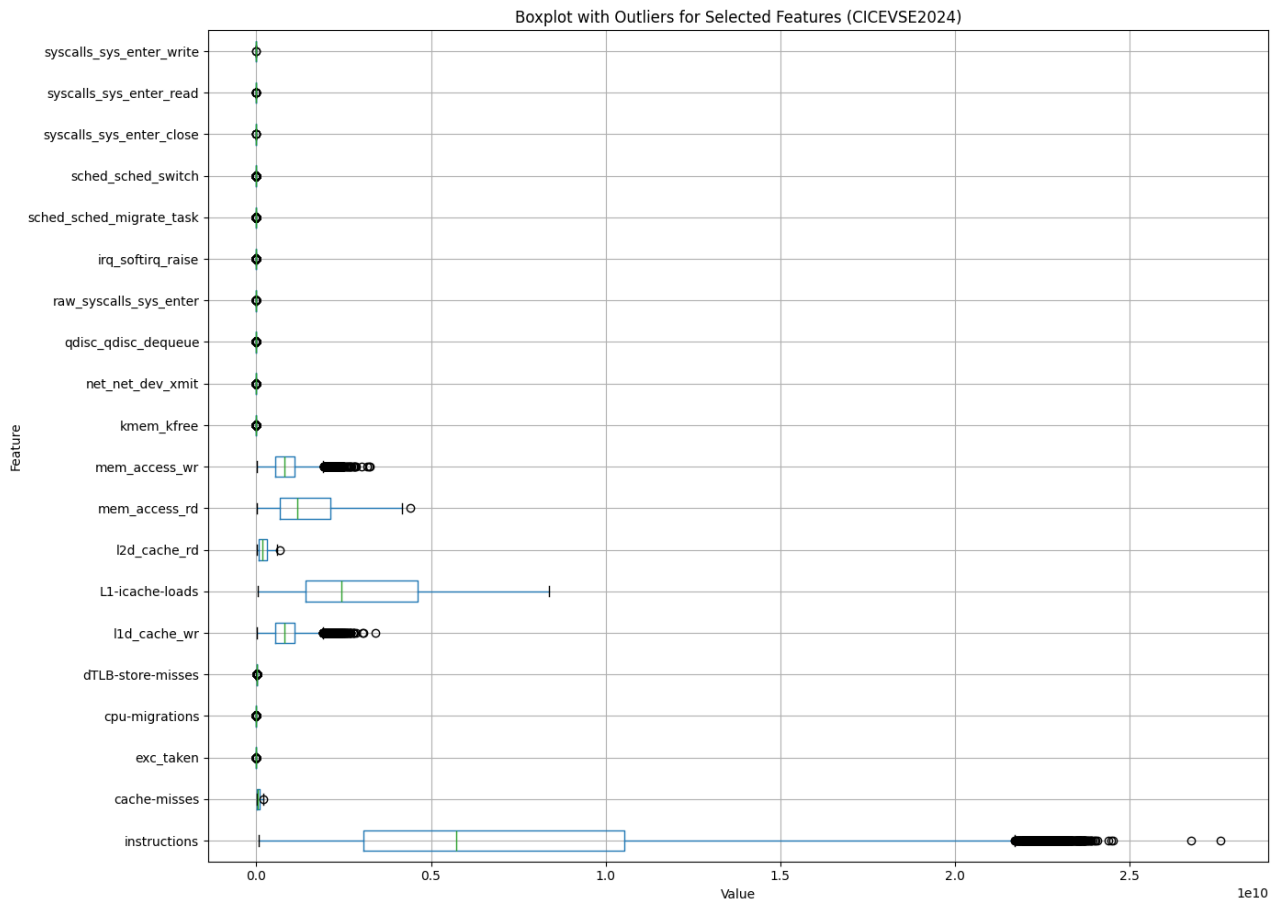


Figure 10: Boxplot of the CICEVSE2024 Dataset Features

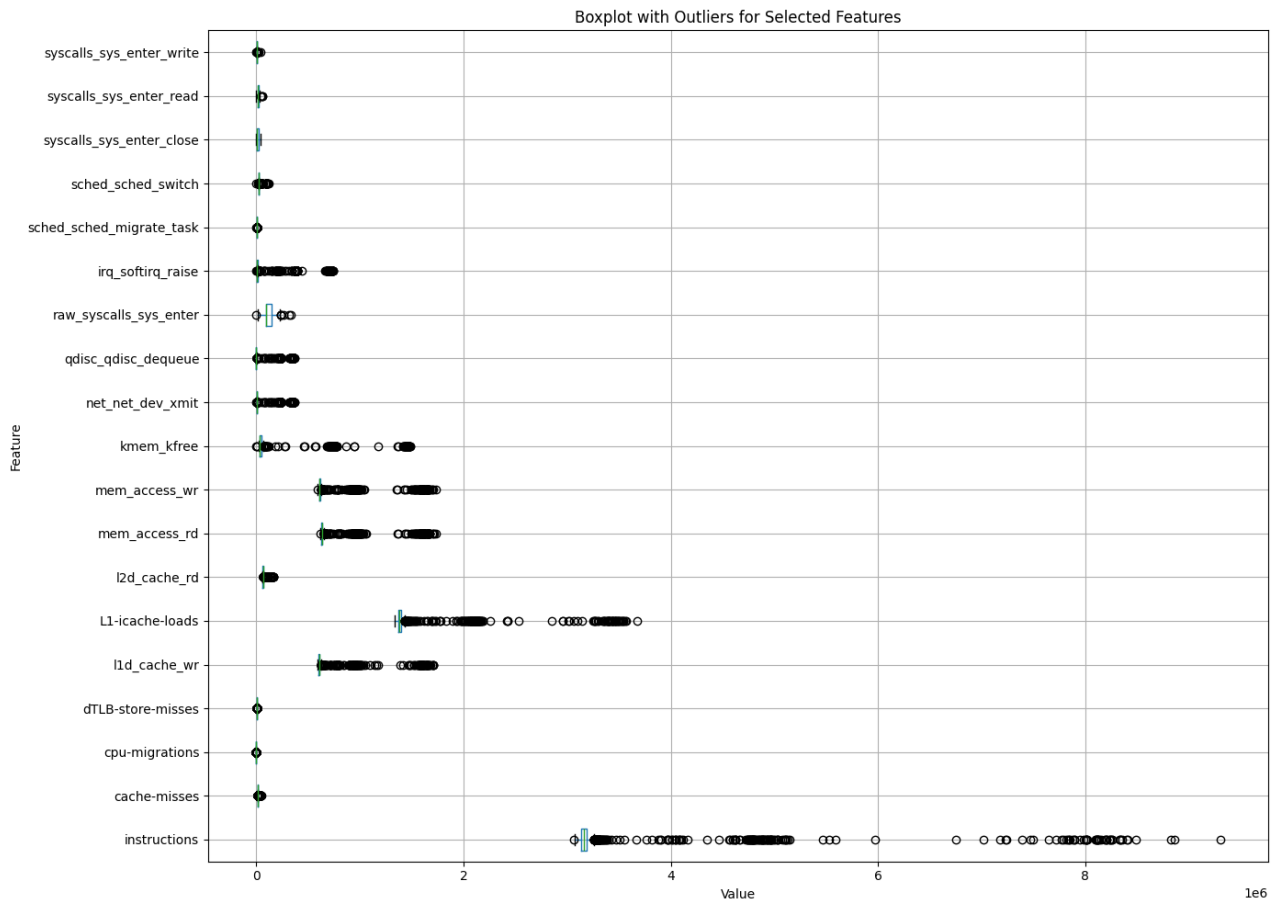


Figure 11: Boxplot of the self-collected Dataset Features

When comparing the two datasets, it becomes apparent that the self-collected dataset exhibits more outliers than the CICEVSE2024 dataset. This is primarily due to the fact that it contains a significantly larger amount of benign data, which makes the attack data stand out more clearly as outliers. Furthermore, the benign data in the self-collected dataset is much quieter and less variable than in the CICEVSE2024 dataset. This reduced variability becomes especially evident in the analysis of individual features, as can be observed in the following graphs using the `L1-icache-loads` feature as an example.



For clarity we will start like before with the selected features from the paper: Enhancing EV Charging Station Security Using A Multi-dimensional Dataset : CICEVSE2024 [EDBF24] the remaining features can be found in the appendix, where you can see similar results to those shown here.

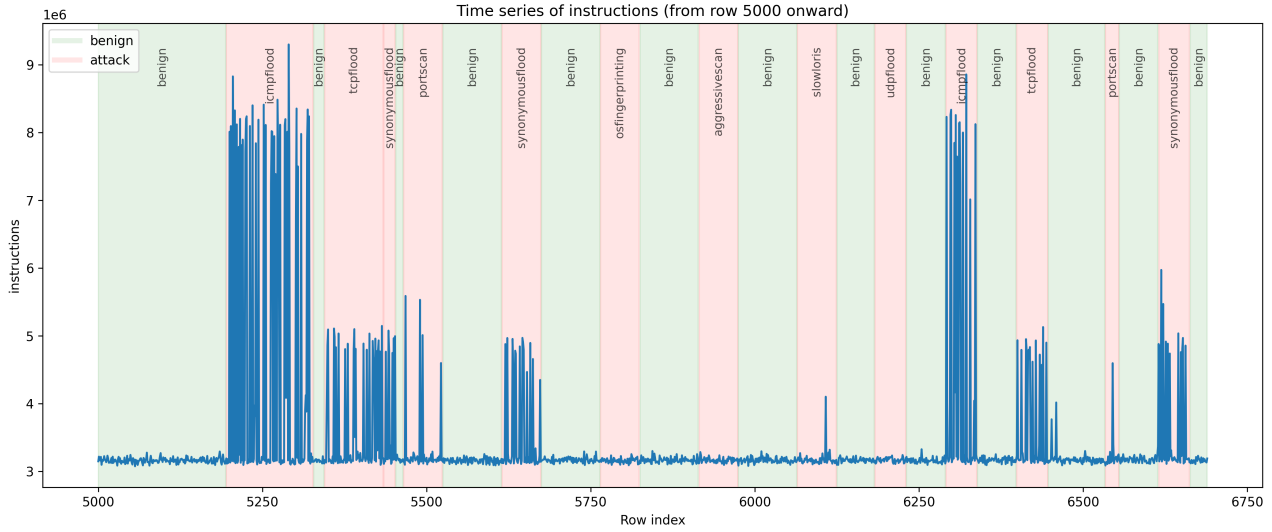


Figure 14: instruction timeline

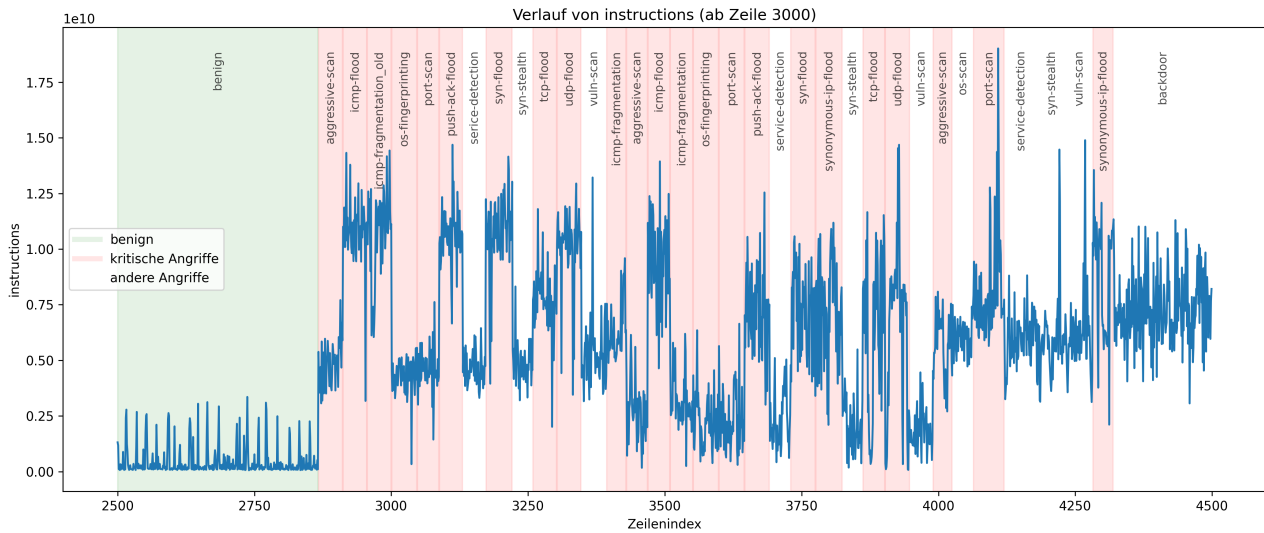
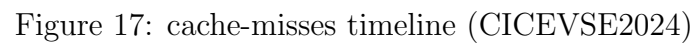


Figure 15: instruction timeline (CICEVSE2024)

The kernel events instruction is used to signal or record noteworthy occurrences within the kernel, such as system calls, interrupts, or state changes, often for the purposes of tracing [comc]. In the CICEVSE2024 dataset, this feature exhibits a relatively stable and low level in benign scenarios. However, in flooding attacks such as ICMP flood, SYN flood, TCP flood, and UDP flood a substantial increase in the kernel events metric can be observed. This increase can be attributed to the fact that such attacks overwhelm the target system, resulting in a significantly higher number of interrupts, state changes, and similar operations compared to benign states or lower-impact attacks, such as OS fingerprinting.



Page 27

preceding aggressive-scan instances show values either comparable to the benign baseline or moderately higher. This irregular pattern may be due to varying background processes or system activity during the respective attack executions. However, this remains speculative, as no comprehensive record of all concurrently running programs in the CICEVSE2024 dataset is available.

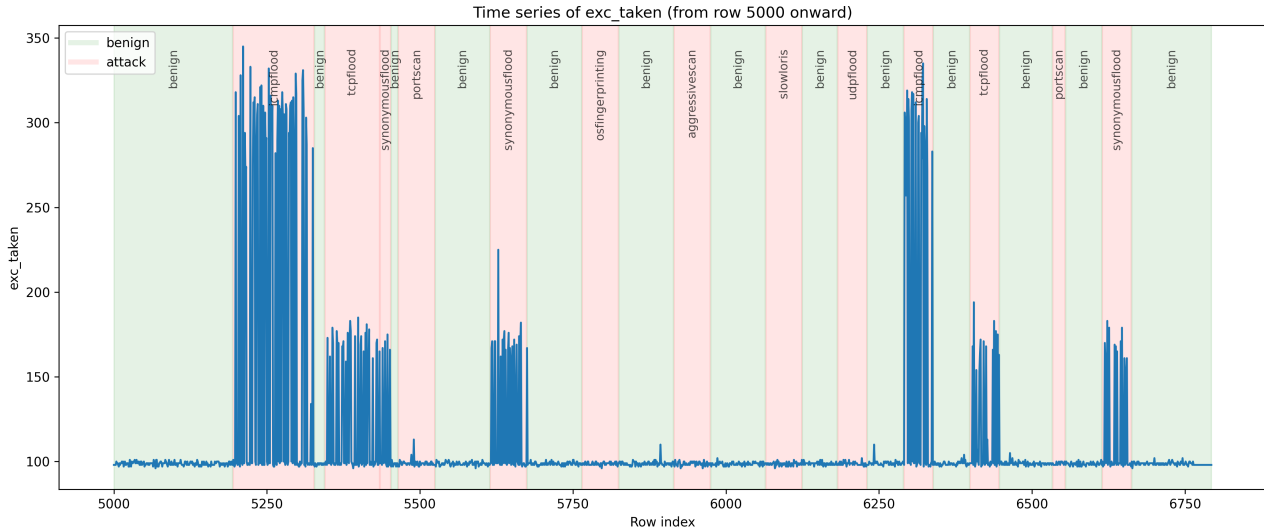


Figure 18: exc-taken timeline

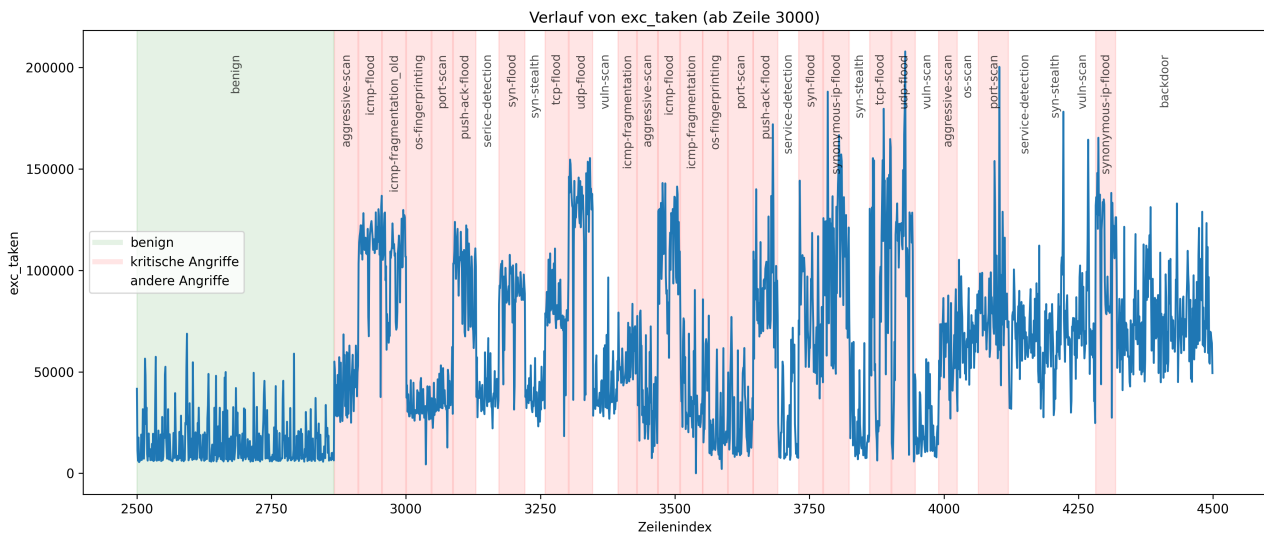


Figure 19: exc-taken timeline (CICEVSE2024)

The *exc_taken* metric represents the number of exceptions taken by the CPU, indicating transitions from normal execution flow to an exception handling routine [lim]. In our measurements, benign scenarios show very low and stable values, while the benign baseline of the

CICEVSE2024 dataset exhibits significantly higher variability. In both datasets, the most pronounced peaks occur during flooding attacks such as ICMP flood, SYN flood, TCP flood, and UDP flood reflecting the increased occurrence of exceptions under high-load conditions.

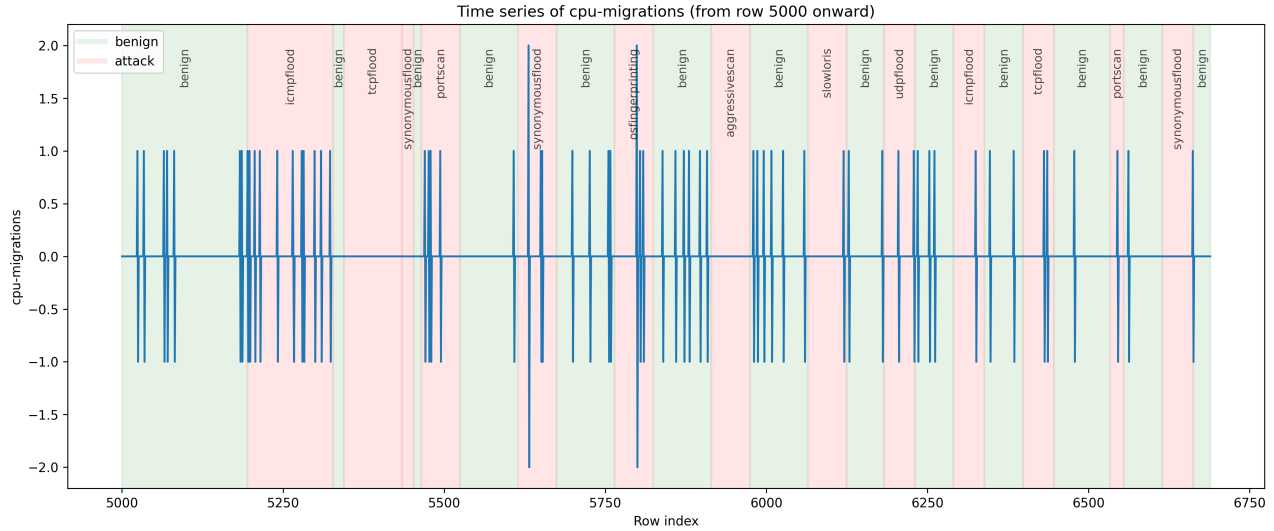


Figure 20: cpu-migrations timeline

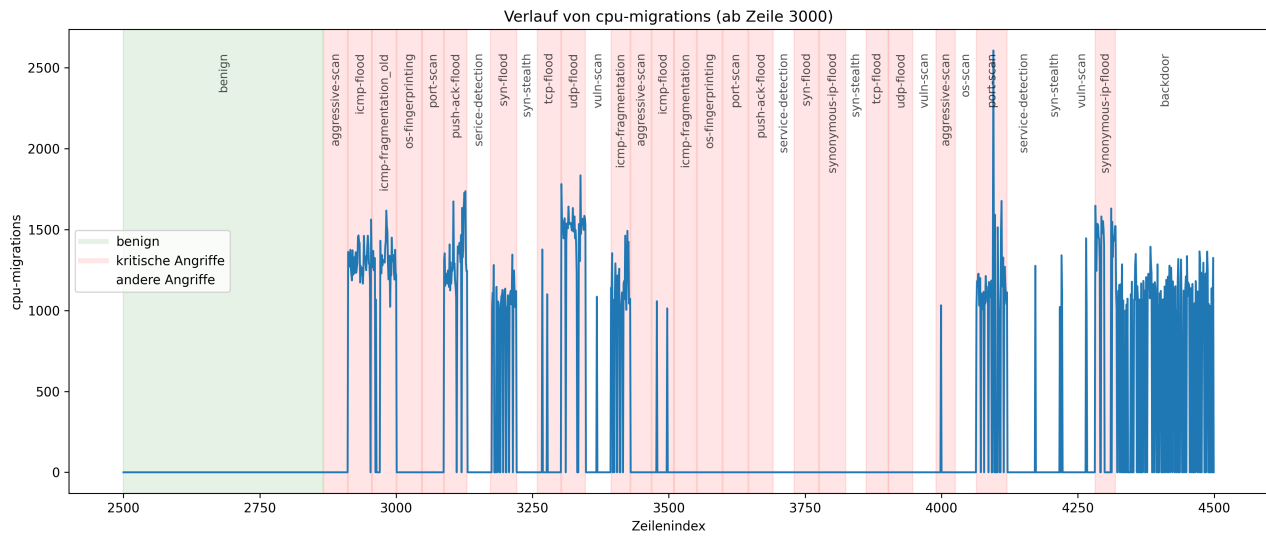


Figure 21: cpu-migrations timeline (CICEVSE2024)

The cpu-migrations metric counts the number of times a process is moved from one CPU core to another, which can occur for load balancing or scheduling purposes [Gre]. In our measurements, this feature shows limited significance, with only occasional peaks during ICMP flood attacks an effect that is not particularly distinct. In contrast, the CICEVSE2024 dataset reports constant zero values in benign scenarios and very pronounced peaks during ICMP flood,

SYN flood, and port scan attacks, making it a potentially valuable indicator for these events. In our data, cpu-migrations appears to behave as a continuously increasing counter, with its reset conditions unclear. Our analysis therefore relies on the difference between consecutive readings, which may contribute to the low discriminative power observed. It is possible—though purely speculative—that the CICEVSE2024 dataset employed a different measurement approach, such as directly logging event counts per sampling interval instead of cumulative totals.

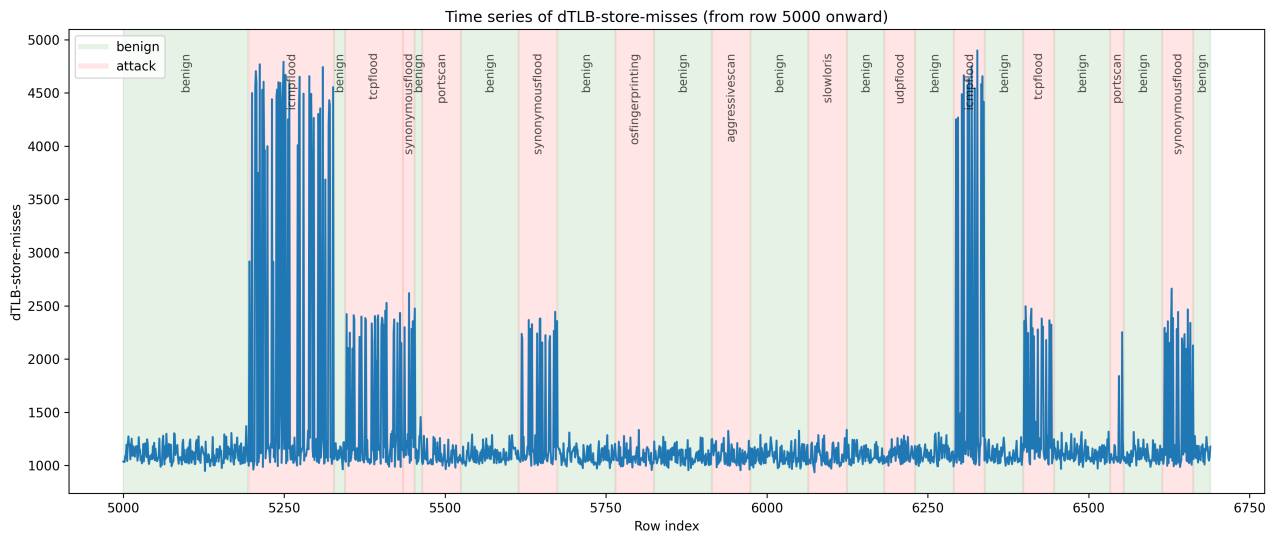


Figure 22: dTLB-store-misses timeline

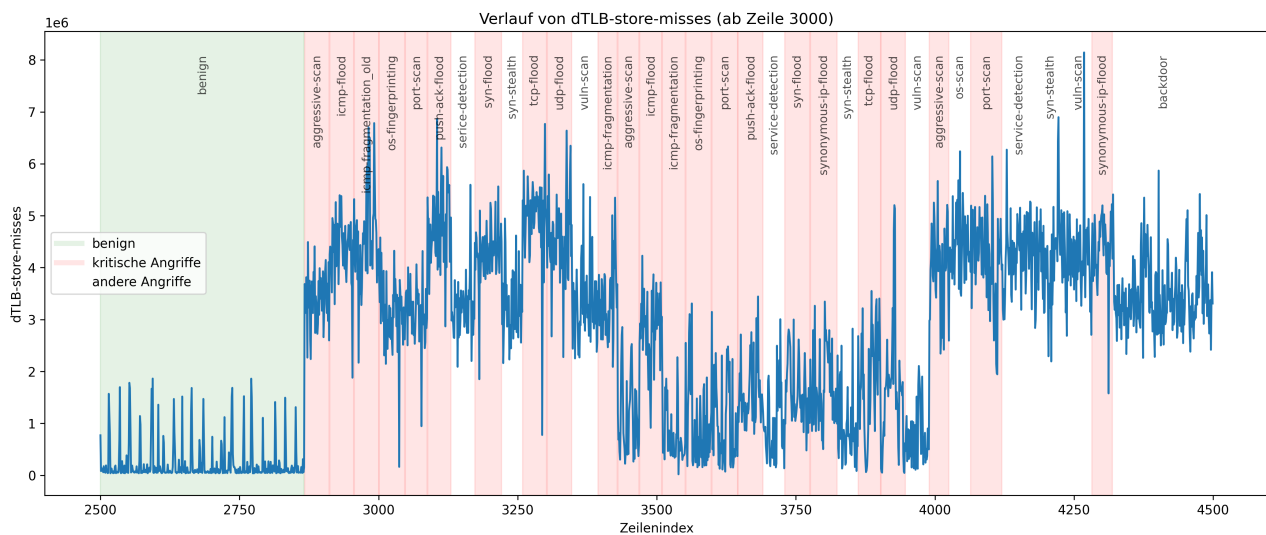


Figure 23: dTLB-store-misses timeline (CICEVSE2024)

The dTLB-store-misses metric measures the number of store (write) operations that miss in the data Translation Lookaside Buffer (dTLB), requiring address translation from slower memory

structures [Dro]. In our measurements, benign scenarios exhibit a very stable and low baseline, whereas the benign baseline of the CICEVSE2024 dataset shows greater variability. In both datasets, the highest peaks occur during flooding attacks such as ICMP flood, SYN flood, TCP flood, and UDP flood. In the CICEVSE2024 dataset, aggressive-scan and port-scan attacks show varying peak intensities: initially strong, then close to benign levels, and later extremely high. This variation may be due to slightly different variants of these attack types being used, although this remains speculative.

Interestingly, flooding attacks such as ICMP flood, SYN flood, TCP flood, and UDP flood consistently produce the highest peaks across multiple performance metrics, including *kernel events*, *cache-misses*, *exceptions taken*, *cpu-migrations*, and *dTLB-store-misses*, as well as in the additional analyzed features. These attacks impose a substantial processing load on the target system, resulting in more frequent interrupts, exceptions, cache lookups, and memory translations, alongside increased process scheduling activity. Among these, the ICMP flood often exhibits the most pronounced impact, which can be attributed to the fact that each ICMP Echo Request must be fully processed and answered, thereby generating significant CPU and memory subsystem activity.

In contrast, benign scenarios especially in our own measurements remain largely stable and low across these features, whereas the CICEVSE2024 dataset displays higher variability in benign baselines. For certain attacks, such as *aggressive-scan* or *port-scan*, the CICEVSE2024 dataset reveals fluctuating peak intensities, which may be due to slight variations in the specific attack implementations used; however, this remains speculative.

6 Feature Selection

6.1 Data Preprocessing Steps

As already mentioned, for the CICEVSE2024 dataset, it was only necessary to remove four rows containing NaN values to obtain clean and usable data for the initial processing step.

For the generated dataset from our environment, I first aimed to include as much benign-status data as possible. Therefore, I made a Prometheus PromQL request to retrieve monitoring data from a night when the environment was not used for other tests (see Appendix ??).

Then, I initiated the attacks to monitor the behavior of the HCP and kernel events in our environment under attack conditions (see Appendix ??). Table 4 provides an overview of all attacks carried out during the experiment.

Table 4: Overview of executed attacks

Attack Type	Tool	Description / Parameters
Slowloris	Python script	500 sockets against target host
UDP Flood	hping3	<code>-flood -udp -p 80</code>
ICMP Flood	hping3	<code>-flood -l</code>
TCP Flood	hping3	<code>-flood -p 80</code>
Synonymous Flood	hping3	<code>-flood -rand-source -p 80</code>
Portscan	nmap	1000 iterations, <code>-sT</code> scan
OS Fingerprinting	nmap	1000 iterations, <code>-O</code> scan
Aggressive Scan	nmap	1000 iterations, <code>-A</code> scan

With the generated log file, it was possible to assign the correct attack intervals to the `dataset.csv` file for machine learning training and data analysis. Table 5 shows the exact time periods used.

Table 5: Attack intervals during data collection

Start Time	End Time	Attack Type
2025-07-23 06:52:35	2025-07-23 07:07:35	Slowloris
2025-07-23 07:27:35	2025-07-23 07:42:35	UDP Flood
2025-07-23 08:25:45	2025-07-23 08:47:50	ICMP Flood
2025-07-23 08:50:49	2025-07-23 09:05:49	TCP Flood
2025-07-23 09:05:49	2025-07-23 09:08:49	Synonymous Flood
2025-07-23 09:10:44	2025-07-23 09:20:44	Portscan
2025-07-23 09:35:44	2025-07-23 09:45:44	Synonymous Flood
2025-07-23 10:00:44	2025-07-23 10:10:44	OS Fingerprinting
2025-07-23 10:25:44	2025-07-23 10:35:44	Aggressive Scan
2025-07-23 10:50:44	2025-07-23 11:00:44	Slowloris
2025-07-23 11:10:21	2025-07-23 11:18:21	UDP Flood
2025-07-23 11:28:21	2025-07-23 11:36:21	ICMP Flood
2025-07-23 11:46:21	2025-07-23 11:54:21	TCP Flood
2025-07-23 12:08:55	2025-07-23 12:12:21	Portscan
2025-07-23 12:22:21	2025-07-23 12:30:21	Synonymous Flood

With these preprocessing steps, a meaningful exploratory data analysis was possible, allowing us to proceed with key feature selection to train the machine learning models for optimal results. This step is essential, as most models do not perform well with 900 or more features. This is also demonstrated in [EDBF24], where the best results were achieved using only 11 to 20 key features.

Furthermore, using too many features would lead to performance issues on the Raspberry Pi in the live system, as it would need to query and process values for up to 900 features every 5 to 60 seconds.

6.2 Feature Selection Using Decision Trees

Feature selection is a crucial aspect in the preparation of modern deep learning algorithms. However, identifying the most effective method that also performs well in a live environment such as an EVSE system with limited hardware resources—remains a challenge.

The paper “Enhancing EV Charging Station Security Using a Multi-dimensional Dataset” [EDBF24]* employed Principal Component Analysis (PCA) to reduce 900 input features for use in an LSTM model. We also experimented with PCA, but due to the nature of this method, all 900 original features must still be requested in the live environment in order to compute the transformed principal components. Executing the required PromQL queries for all 900 features caused the CPU usage on our Raspberry Pi setup to average nearly 50 %, which makes this approach

unsuitable for real-time deployment.

Following this limitation, we were compelled to explore alternative feature selection techniques that could offer high performance without requiring access to the full feature set. The paper *“Comparative Analysis of Feature Selection Techniques for LSTM-Based Network Intrusion Detection Models”* [Kot24]* evaluated multiple feature selection strategies to determine the most effective combinations for LSTM-based algorithms. According to their findings, the

Model Configuration	Accuracy	F1 Score
LSTM + Without Feature selection	85%	0.872
LSTM + PCA	76%	0.821
LSTM + Univariate Feature Selection	78%	0.836
LSTM + Recursive Feature Elimination	78%	0.837
LSTM + L1- Based Feature Selection	77%	0.829
LSTM + Select Best Feature Selection	83%	0.871
LSTM + Tree Based Feature Selection	82%	0.857
LSTM + Sequential Feature Selection	69%	0.736

Figure 24: Comparison of feature selection methods for LSTM accuracy [Kot24].

LSTM model without any feature selection achieved the highest accuracy at 85 %. However, as previously mentioned, using all 900 features is not feasible in our hardware-constrained environment. The next best accuracies were achieved using LSTM combined with either the Select Best Feature Selection method or Tree-Based feature selection, with only a 1 % accuracy drop and an F1-score difference of just 0.014.

Given this background, we decided to adopt the Decision Tree algorithm for feature selection. This allows us to identify the most relevant features while maintaining a high level of accuracy, and keeping resource usage within our systems constraints.

With this information, I created a Decision Tree to select the most relevant features for real-time detection in our EVSE system. I used the `DecisionTreeClassifier` from the `sklearn.tree` module, which is easy to use and integrates well with common data science workflows.

In this step, our preprocessed dataset was used to train a Decision Tree Classifier for feature selection. First, the categorical labels were converted into a binary format, where `benign` was mapped to 0 and `attack` to 1. To prevent data leakage, the columns `Label`, `timestamp`, and `attack` were dropped from the feature set before training. The Decision Tree Classifier was then trained on the remaining features using a fixed random seed (`random_state=42`). This model primarily served to identify the most relevant features for subsequent analysis and classification tasks.

This approach allowed us to significantly reduce the number of input features while preserving high detection accuracy and runtime performance.

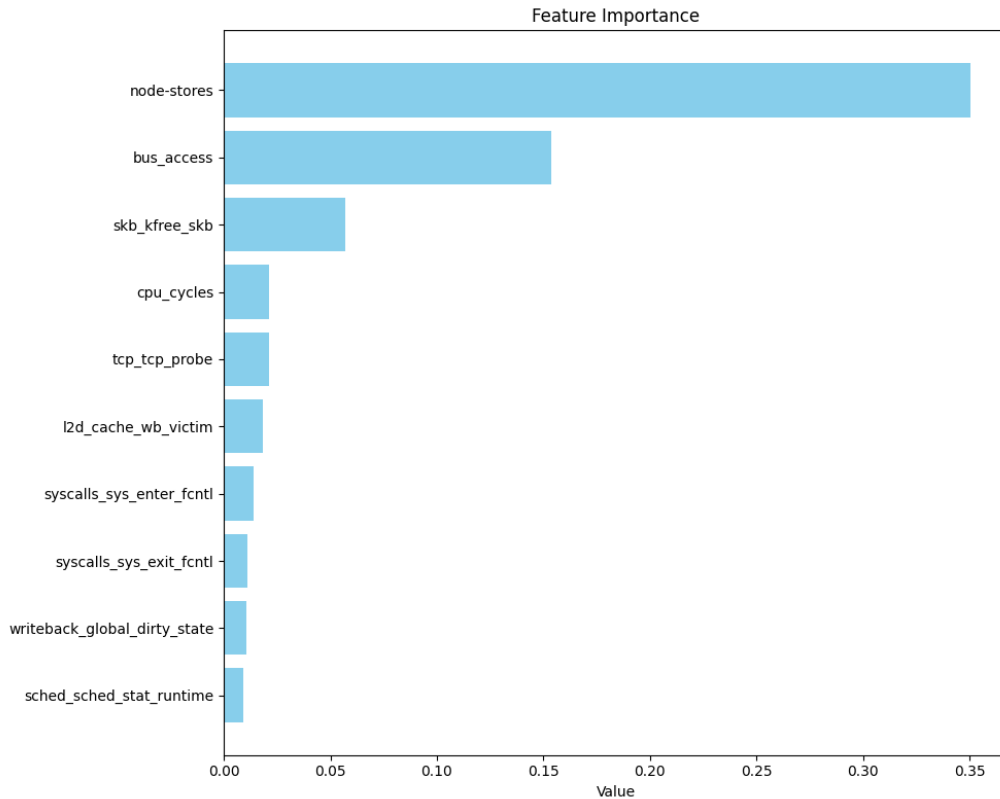


Figure 25: Top 10 most important features selected by the Decision Tree.

node-stores: This event refers to memory store operations targeting specific NUMA nodes in a multi-node system. Monitoring node-store events helps analyze memory access patterns and data locality [comd], which can be important to detect unusual memory write activity often seen during attacks like **UDP Floods** or **ICMP Floods** where large volumes of data are rapidly written.

bus-access: This event tracks accesses on the system bus, responsible for communication between CPUs, memory, and peripheral devices [comb]. Elevated or abnormal bus-access patterns can indicate intensive network or port scanning activities such as **Portscans** or **Aggressive Scans** that generate many small requests to different system resources.

skb_kfree_skb: This tracepoint event in the Linux kernel occurs when socket buffers (SKBs) managing network packets are freed [Skb]. Tracking this event is crucial for detecting attacks that flood or manipulate network connections, including **Slowloris** (many half-open connections), **SYN Floods** (half-open TCP connections), **UDP Floods**, and **ICMP Floods**, because these attacks cause unusually high packet processing and freeing activity.

Using the 20 most important features identified by the Decision Tree, I created a correlation matrix to examine feature redundancy.

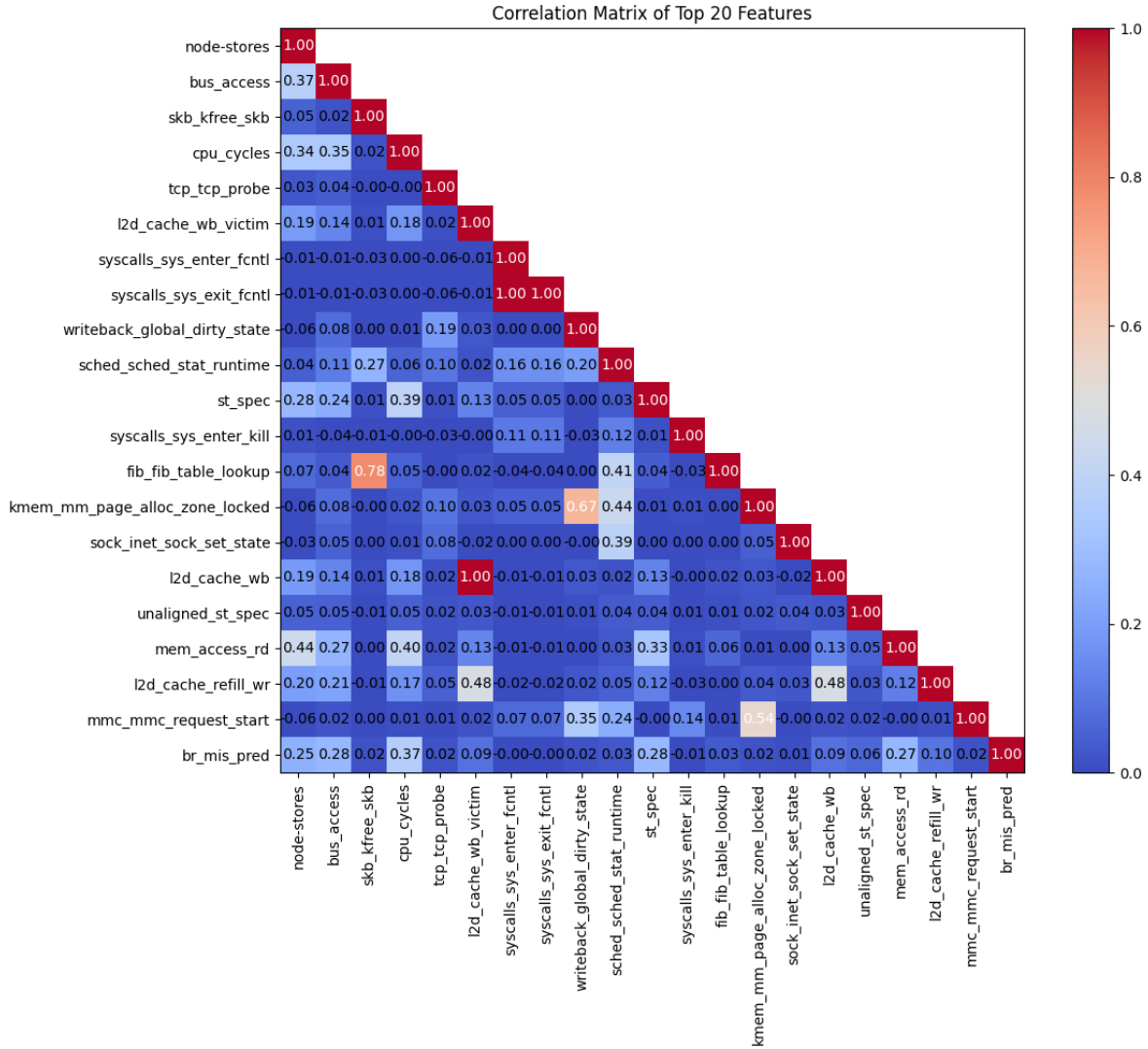


Figure 26: Correlation matrix top Features with Decision Tree.

The correlation matrix in Figure 26 shows that there is almost no correlation between the top 20 features selected by the Decision Tree. This result is highly desirable, as it indicates that the chosen features are largely independent from one another and therefore provide complementary information rather than redundant signals. In practice, this means that the model can learn from a broad spectrum of system behaviors instead of being biased toward multiple highly correlated indicators of the same phenomenon.

For our LSTM-based detection model later, this independence between features is particularly beneficial. LSTMs are designed to capture temporal dependencies and patterns in sequential data, and providing them with a diverse set of non-redundant input signals increases their ability to generalize across different attack types and benign conditions [EJP23]. Furthermore, reducing feature correlation improves computational efficiency, as the system does not waste resources processing multiple overlapping signals. Overall, the low correlation among the

selected features represents an optimal foundation for training our LSTM model in a resource-constrained environment.

If we apply the same Decision Tree Classifier to the CICEVSE2024 dataset, we naturally obtain different results, as the features exhibit different peaks and benign data patterns, which we had already observed earlier.

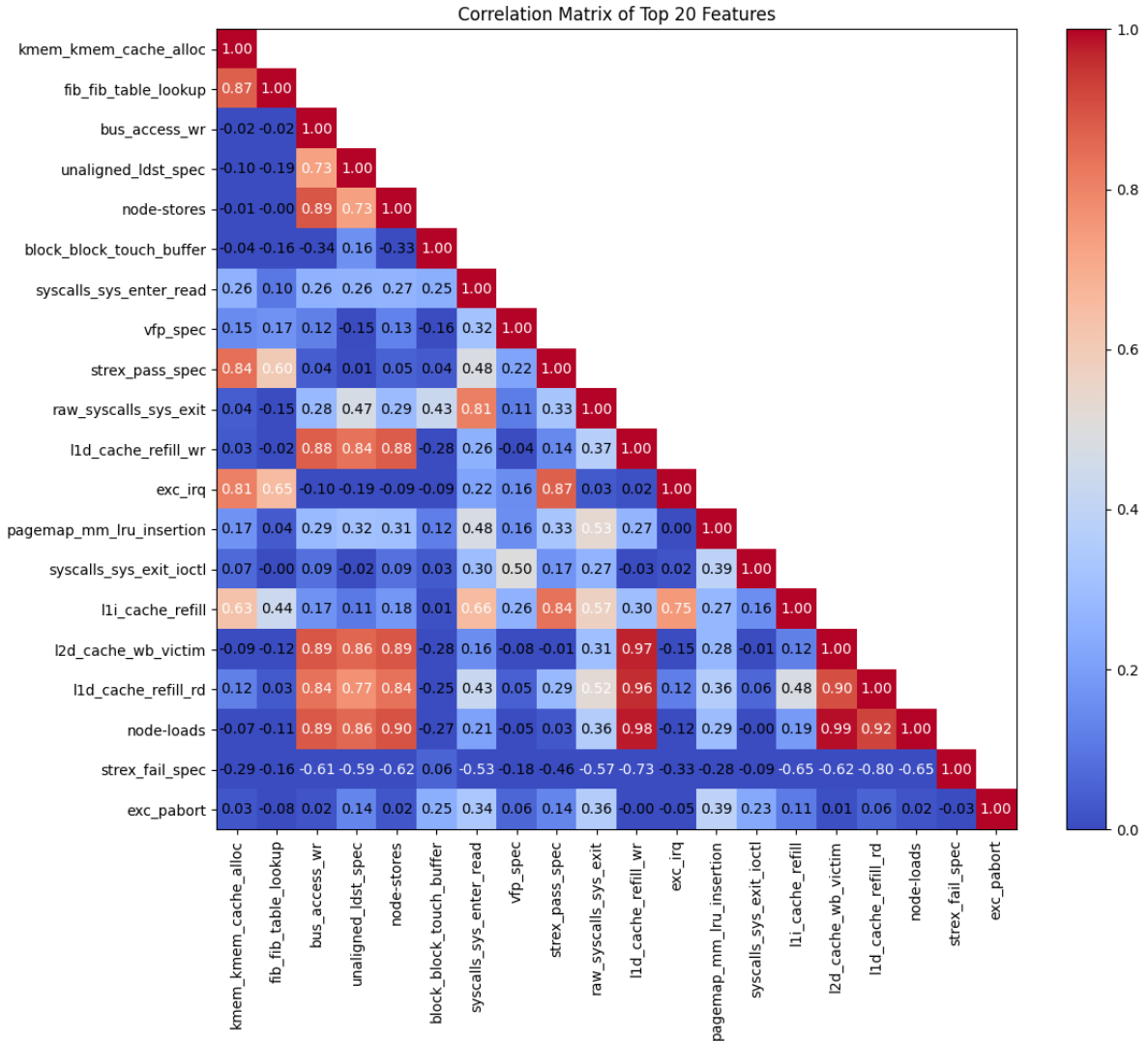


Figure 27: Correlation matrix of top features selected by the Decision Tree on the CICEVSE2024 dataset.

Nevertheless, we can still observe some correlations among the selected features. For example, in both datasets, features such as `node-stores`, `l2d_cache_wb_victim` and `fib_fib_table_lookup` were consistently selected. Similarly, the `skb_kfree_skb` and `fib_fib_table_lookup` features also show a strong correlation, which is particularly evident in the next figure.

Nevertheless, we can still observe some correlations among the selected features. For example, in

both datasets, features such as `node-stores`, `l2d_cache_wb_victim`, and `fib_fib_table_lookup` were consistently selected. The `l2d_cache_wb_victim` event represents write-back operations of the Level 2 data cache for evicted “victim” lines, indicating high memory traffic due to cache evictions under intensive load conditions, such as flooding attacks. The `fib_fib_table_lookup` event is triggered during routing table lookups in the kernel’s Forwarding Information Base, reflecting increased network routing activity common in scanning or flood attacks. Similarly, the `skb_kfree_skb` and `fib_fib_table_lookup` features also show a strong correlation, which is particularly evident in the next figure. The `skb_kfree_skb` event occurs when socket buffers managing network packets are freed, a critical indicator of network connection flooding or manipulation seen in attacks like Slowloris, SYN Floods, UDP Floods, and ICMP Floods. These interrelations highlight complementary aspects of network and memory behavior under attack scenarios.

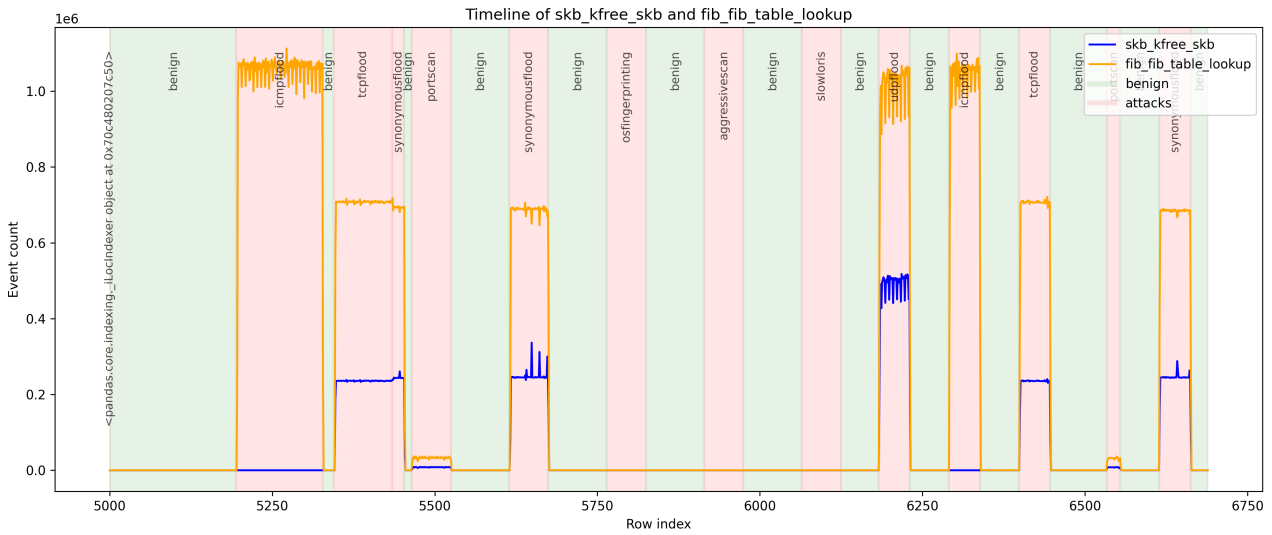


Figure 28: Correlation between `skb_kfree_skb` and `fib_fib_table_lookup` features.

So far, we have observed that the Decision Tree Classifier selects similar features in both datasets. This indicates that it is likely to select comparable features in other, similarly structured setups, even when more than 900 potential features are available. However, the strongest predictive power is concentrated in only a small subset of these features.

Nevertheless, it is essential to examine the selected features in the context of the timeline in order to better understand their behavior and patterns over time, and to assess whether they are useful for the LSTM algorithm in detecting attacks. In this work, we do not provide a detailed discussion of every individual feature; instead, all corresponding figures can be found in the appendix. Here, we focus on tree representative features.

As already observed, the `node-stores` feature shows a clear pattern that helps in detecting

flooding attacks, as described before, while the `sys_calls_sys_enter_kill` feature records the number of times the Linux kernel processes the “kill” system call (i.e., `sys_kill`), which is used by processes to send signals to other processes. During a port scan attack, tools such as `nmap` or similar scanners attempt to rapidly open and close many connections across a wide range of ports. This aggressive probing leads to the frequent creation and termination of network sockets and, in some cases, the spawning of short-lived processes for each connection attempt. In summary, the high peaks in `sys_calls_sys_enter_kill` during port scan attacks reflect the system’s intensive signaling behavior as it deals with the rapid creation and destruction of connections and processes, making it a strong indicator for detecting such attacks.

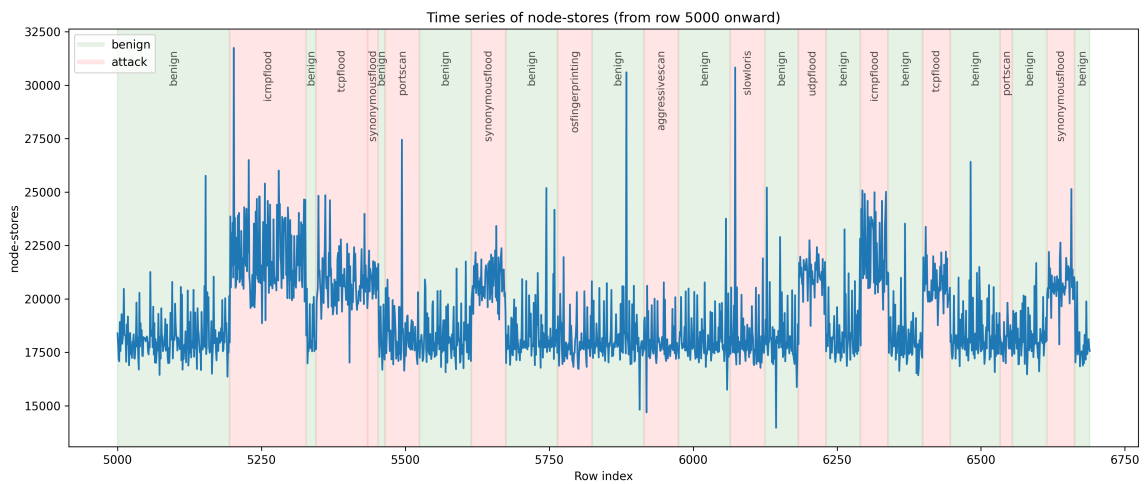


Figure 29: Timeline behavior of the `node-stores` feature, showing distinguishable patterns during flooding attacks.

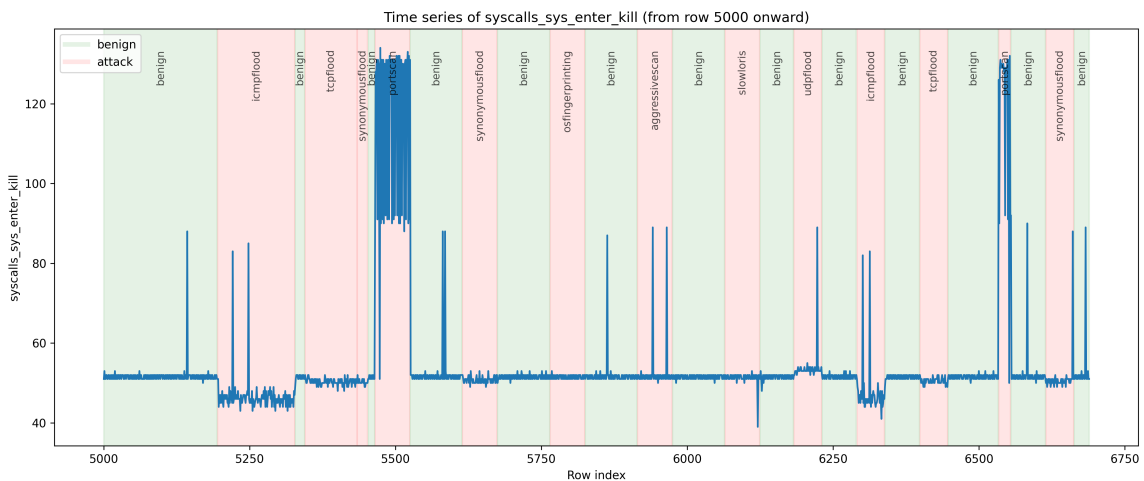


Figure 30: Timeline behavior of the `sys_calls_sys_enter_kill` feature, highlighting its usefulness in detecting port scan attacks.

On the other hand, some features are unfortunately not very useful for attack detection, as

they show almost no distinguishable peaks between benign and attack data. This is the case for the feature `unaligned_st_spec`, as illustrated in the figure below.

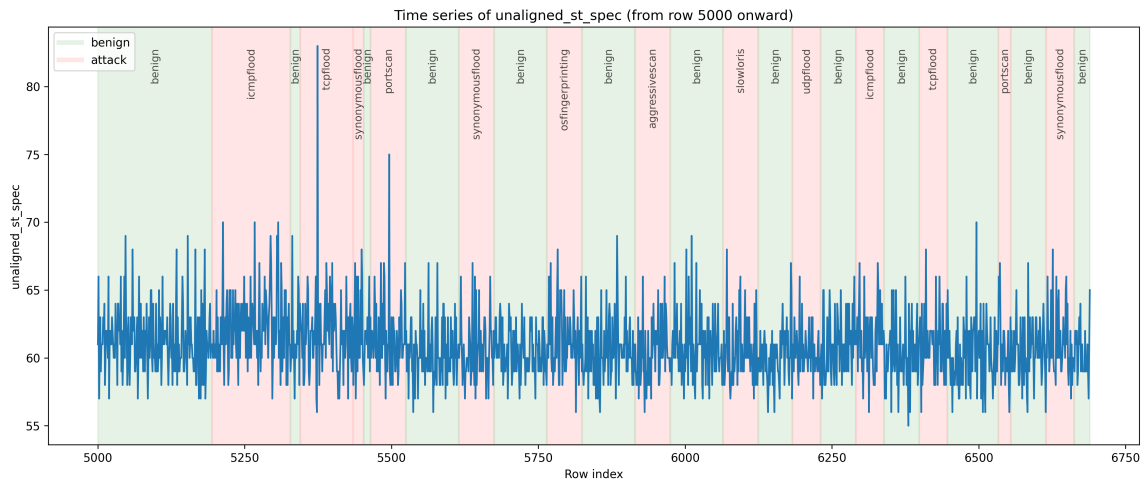


Figure 31: Timeline behavior of the `unaligned_st_spec` feature, showing little variation between benign and attack data.

7 LSTM Architecture: Structure, Training, and Optimization

7.1 LSTM Architecture and Preprocessing

A foundational understanding of the structure and principles of Long Short-Term Memory (LSTM) networks has already been provided in Chapter 2_Fundamentals. In this section, I will delve deeper into the specific architecture of the LSTM used in this work and highlight the importance of time-step-based data analysis in this context.

Time-step analysis offers a major advantage of LSTMs over other deep neural network (DNN) models, particularly in the detection of cyber attacks or anomalies. Most such events do not occur abruptly. Instead, they often emerge gradually over time [Rah+25]. LSTM networks are specifically designed to recognize temporal patterns and sequential dependencies, making them especially well-suited for identifying these slow-developing behaviors.

In order to detect time-dependent attacks effectively, the training data must be well-organized so that the models can learn time-relevant patterns—both during training and later in the live system. Therefore, the preprocessing steps used to split the data into training and test sets must be carefully structured and should not be randomized.

At this stage, the data has generally already been cleaned, as described in Chapter 6_Feature Selection.

In my case, I split the dataset into 10 chronological blocks. Within each block, the data is sorted by time steps. Then I combined 7 of these blocks to the training dataset and used the remaining 3 for the test dataset. I ensured that both sets had approximately the same distribution of the labels `benign` and `attack`, to prevent class imbalance.

The following listing shows the exact implementation of the data-splitting and label distribution verification process:

```

1
2 import pandas as pd
3
4 # Load the preprocessed dataset
5 df = pd.read_parquet('preprocessed_custom_data.parquet')
6
7 # Split the dataset into 10 time-ordered blocks
8 block_size = len(df) // 10
9 blocks = [df.iloc[i * block_size:(i + 1) * block_size] for i in range(9)]
10 blocks.append(df.iloc[9 * block_size:]) # Last block takes the remaining
    data
11
12 # Print class distribution in each block
13 label_counts = [block['Label'].value_counts() for block in blocks]
14 for i, counts in enumerate(label_counts):
15     print(f'Block {i + 1}:', dict(counts))

```

```

16
17 # Use blocks 2, 5, and 8 as test data, the rest as training data
18 test_blocks = [blocks[1], blocks[4], blocks[8]]
19 train_blocks = [block for i, block in enumerate(blocks) if i not in [1,
    4, 8]]
20
21 # Concatenate the blocks to form final datasets
22 test_df = pd.concat(test_blocks).reset_index(drop=True)
23 train_df = pd.concat(train_blocks).reset_index(drop=True)
24
25 # Save the resulting datasets as Parquet files
26 test_df.to_parquet('test_block.parquet')
27 train_df.to_parquet('train_block.parquet')
28
29 # Reload saved Parquet files (for verification or further use)
30 train_df = pd.read_parquet('train_block.parquet')
31 test_df = pd.read_parquet('test_block.parquet')
32
33 # Function to compute label distribution percentages
34 def label_percentages(df):
35     return (df['Label'].value_counts(normalize=True) * 100).round(2)
36
37 # Display label distribution
38 print("Train Set:")
39 print(label_percentages(train_df))
40
41 print("\nTest Set:")
42 print(label_percentages(test_df))

```

Listing 1: Block-wise time-ordered split of the dataset for training and testing, including class distribution balancing

7.2 Parameter Configuration and Hyperparameter Optimization

In this Chapter we take a closer look at the chosen structure from the LSTM architecture and explain it.

```

1 from config_feature_test import FEATURES, LABEL
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4
5 # Load training and test datasets
6 train_df = pd.read_parquet('train_block.parquet')
7 test_df = pd.read_parquet('test_block.parquet')

```

Listing 2: Loading training and test datasets, and preparing feature and label configuration

```

# Function to compute label distribution percentages
def label_percentages(df):
    return (df['Label'].value_counts(normalize=True) * 100).round(2)

# Display label distribution
print("Train Set:")
print(label_percentages(train_df))

print("\nTest Set:")
print(label_percentages(test_df))

```

```

Train Set:
Label
benign    86.82
attack    13.18
Name: proportion, dtype: float64

Test Set:
Label
benign    84.93
attack    15.07
Name: proportion, dtype: float64

```

Figure 32: Balanced label distribution between the training and test datasets.

In the text code listing we label the data in the dataset and scale it from 0 to 1 this is especially in the LSTM model important because the unscaled data can not be readed well from the model.

In the code listing 3, we label the data by converting the class labels from strings (“benign”, “attack”) into binary values (0 and 1), and we scale all feature values using the **StandardScaler**. This scaling step is crucial for training an LSTM model effectively.

Unscaled data can lead to numerical instability and poor convergence in neural networks. LSTMs are particularly sensitive to the scale of input features, because they use sigmoid (explained later in detail) activation functions, which operate best when input values are within a small, centered range—typically between -1 and 1. If the features have very different magnitudes, such as bytes transmitted (e.g. in thousands) versus CPU usage (e.g. in fractions), the model may focus disproportionately on features with larger numeric ranges, leading to biased learning and suboptimal performance [Coo+17].

By standardizing the input features (i.e., transforming them to have zero mean and unit variance), we ensure that the LSTM model can learn temporal dependencies more effectively and converge faster during training.

```

1 # Convert labels to binary values: 'attack' -> 1, 'benign' -> 0
2 train_df[LABEL] = train_df[LABEL].map({'attack': 1, 'benign': 0})
3 test_df[LABEL] = test_df[LABEL].map({'attack': 1, 'benign': 0})
4
5 # Separate features and labels
6 X_train = train_df[FEATURES] # Select defined features for training

```

```

7  y_train = train_df[LABEL]      # Select the corresponding labels
8  X_test = test_df[FEATURES]    # Select same features for testing
9  y_test = test_df[LABEL]      # Select the corresponding labels
10
11 # Scale features using StandardScaler (fit on training data only)
12 scaler = StandardScaler()
13 X_train_scaled = scaler.fit_transform(X_train) # Fit and transform on
    training data
14 X_test_scaled = scaler.transform(X_test)      # Transform test data
    using same scaler

```

Listing 3: Preprocessing: Label encoding, feature selection and scaling for LSTM training

It was also necessary to train the LSTM model with time sequences in order to fully leverage the advantages of LSTM architectures. Since LSTMs are specifically designed to recognize temporal patterns, providing input in sequential form significantly improves performance. Additionally, it is important that each sequence is labeled consistently as either *benign* or *attack*. This avoids ambiguous target values during training and ensures that the model can learn clear and unambiguous patterns for classification.

```

1  import numpy as np
2
3  def create_sequences(X, y, window_size=10):
4      X_seq = []
5      y_seq = []
6      for i in range(len(X) - window_size + 1):
7          label_window = y[i:i+window_size]
8
9          # Only keep sequences where all labels in the window are the same
10         if np.all(label_window == label_window[0]):
11             X_seq.append(X[i:i+window_size]) # Add sequence window of
                features
12             y_seq.append(label_window[0])    # Add corresponding
                consistent label
13     return np.array(X_seq), np.array(y_seq)

```

Listing 4: Create labeled sequences from time-series data for LSTM training

The following listing is particularly interesting, as it highlights several important configuration choices that warrant further discussion.

Component	What it is	Why it is used
Dropout	A regularization technique that randomly sets a fraction of input units to zero during training.	Dropout reduces overfitting by randomly disabling parts of the model during training, which leads to a more robust and generalized model.
Sigmoid	An activation function that outputs values between 0 and 1.	Because it is ideal for binary classification (benign, attack) by producing probabilities for class 0 or 1.
Binary Cross-Entropy	A loss function for binary classification tasks.	Measures the information loss between predicted probabilities and true binary labels, reflecting both class separation and probability calibration.
Adam	An optimization algorithm that combines momentum and adaptive learning rates.	Enables fast convergence and performs well with noisy and sparse gradients by adaptively scaling learning rates for each parameter.

Table 6: Explanation of key components used in the LSTM model configuration. [Aro+21][Sae][Ram+18][KB15]

```

1  from tensorflow.keras.models import Sequential
2  from tensorflow.keras.layers import LSTM, Dense, Dropout
3
4  # Function to create a stacked LSTM model
5  def create_lstm_model(num_features):
6      model = Sequential([
7          # First LSTM layer (returns full sequences for stacking)
8          LSTM(30, input_shape=(10, num_features), return_sequences=True),
9          Dropout(0.2), # Dropout to prevent overfitting
10         # Second LSTM layer (only returns final output)
11         LSTM(15),
12         Dropout(0.2), # Additional regularization
13         # Output layer for binary classification
14         Dense(1, activation='sigmoid')
15     ])
16
17     # Compile model with binary crossentropy loss and accuracy metric
18     model.compile(
19         loss='binary_crossentropy',
20         optimizer='adam',
21         metrics=['accuracy']

```

```

22     )
23     return model
24
25     # Create model with number of selected input features
26     model = create_lstm_model(len(FEATURES))
27
28     # Print model summary
29     model.summary()
30
31     # Print number of selected features
32     print(len(FEATURES))

```

Listing 5: LSTM model definition for time-series classification

do I have to explain `batch_size`. Hat ja nicht wirklich einfluss auf das trainingsergebnis. In addition, it should be noted that an early stopping function can be helpful in some cases instead of using a fixed number of 20 epochs. However, for simplicity, I do not discuss or explain the additional code in this work.

```

1  # Train the LSTM model using the training sequences
2  history = model.fit(
3      X_train_seq,          # Input training sequences
4      y_train_seq,          # Corresponding labels
5      epochs=20,            # Number of times the model will iterate over the
                           entire training data
6      batch_size=32,        # Number of samples processed before the model
                           updates
7      validation_data=(X_test_seq, y_test_seq), # Data for evaluating the
                           model during training
8      verbose=1             # Displays progress bar and training metrics
9  )

```

Listing 6: Training the LSTM model with time-sequenced data

After gaining an understanding of how we designed the LSTM architecture for our use case and the meaning of its various functions, we will review the results in the next chapter.

8 Analysis and Evaluation

8.1 Experimental Setup and Evaluation Metrics

As mentioned in Chapter 7.1 *LSTM Architecture and Preprocessing*, the data was split into two datasets: one for training and one for testing. In the following figures, we analyze the trained model using the test dataset, which is also included in the appendix.

At first we tested the lstm architecutre in the test dataset where we took befor a closer look, that the distribution of attack an benign data is similar to the train data. the result where plotted with an confusion matrix.

In the following, we tested the LSTM model in the live system too. Using the following scripts:

```

1  csv_file = 'monitor_lstm_predictions.csv'
2  header = ['timestamp'] + FEATURES + ['benign_prob', 'attack_prob', '
    prediction']
3
4  # Function to query a single metric value from Prometheus
5  def get_metric_value(prom, feat):
6      base_feat = feat.replace('-', '_')
7
8      # Try without "perf_" prefix
9      try:
10         data = prom.get_current_metric_value(metric_name=f"sum({base_feat}
            )")
11         df = MetricSnapshotDataFrame(data)
12         if not df.empty and 'value' in df.columns:
13             return float(df.iloc[0]['value'])
14     except Exception:
15         pass
16
17     # Try with "perf_" prefix as fallback
18     try:
19         data = prom.get_current_metric_value(metric_name=f"sum(perf_{
            base_feat})")
20         df = MetricSnapshotDataFrame(data)
21         if not df.empty and 'value' in df.columns:
22             return float(df.iloc[0]['value'])
23     except Exception:
24         pass
25
26     return np.nan

```

Listing 7: Create CSV file and define the correct PromQL query

```

1  while True:
2      # 1. Query metric values

```

```

3     values_array = [get_metric_value(prom, feat) for feat in FEATURES]
4
5     # 2. Log to CSV
6     row = [datetime.now().isoformat()] + values_array + [benign_prob,
7         attack_prob, prediction]
8     with open(csv_file, 'a', newline='') as f:
9         csv.writer(f).writerow(row)
10
11     time.sleep(5)

```

Listing 8: Get metric values and log them into the CSV file

At the same time, I started the attack script—just [listing 17] as used before—to automate the attacks and generate a synchronized timestamp log. This allows later combination into a single CSV file, making the exploratory data analysis more effective. You can find this dataset also attached under the name: "monitor_lstm_predictions_livedata.parquet".

8.2 Results of Anomaly Detection Using the Machine Learning Model

In the chapter *Exploratory Data Analysis*, we also took a closer look at several events that are mentioned in the paper [EDBF24]. These events were well suited for comparison with our dataset. During the analysis, we observed that the highest variance occurred during flooding attacks, while port scans showed almost no changes in the selected events.

Now, we also examine the events identified as key features by the decision tree algorithm to determine whether we can uncover additional findings.

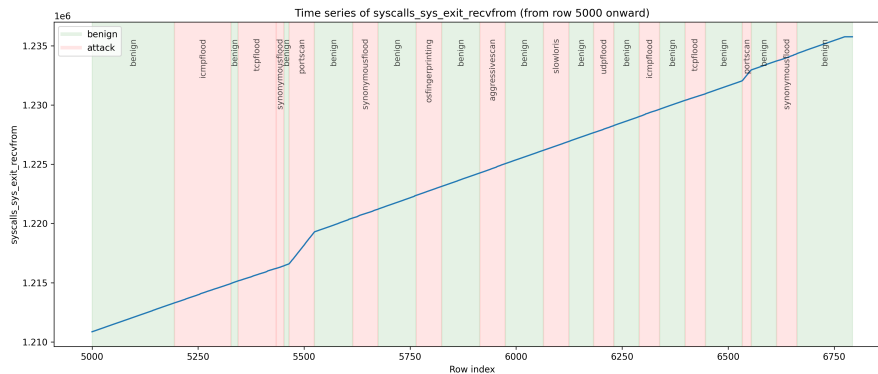


Figure 33: syscalls_sys_exit_recvfrom timeline

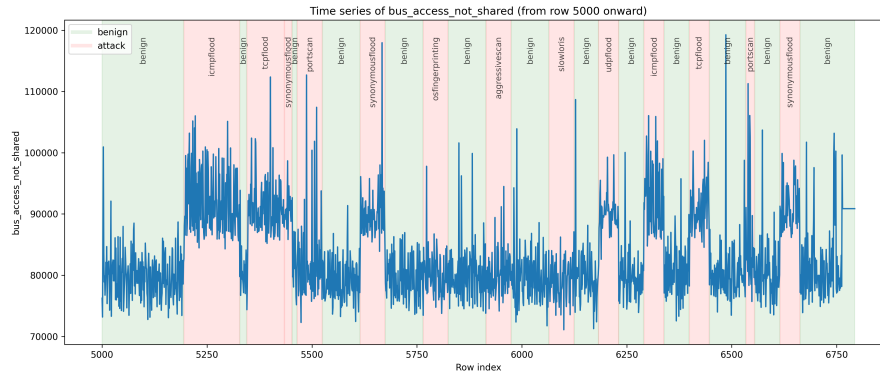


Figure 34: bus_access_not_shared timeline

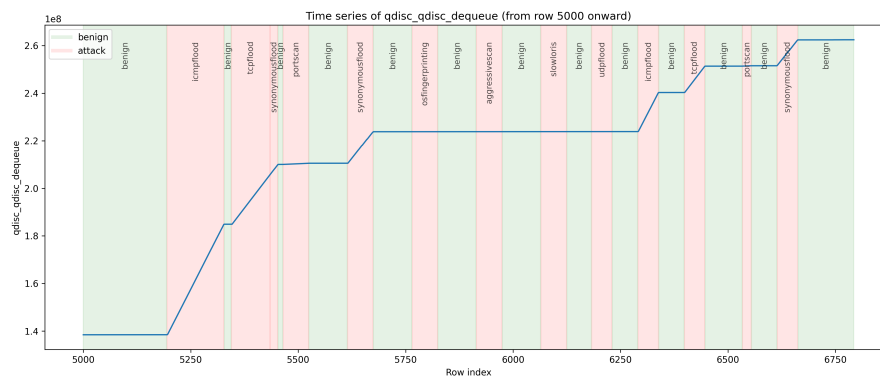


Figure 35: qdisc_qdisc_dequeue timeline

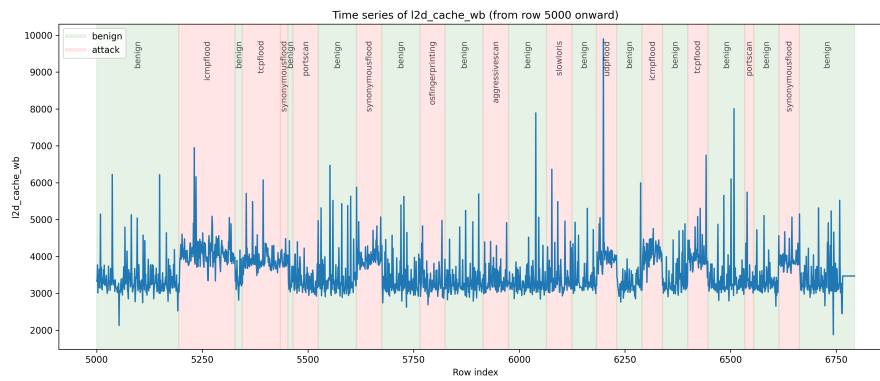


Figure 36: l2d_cache_wb timeline

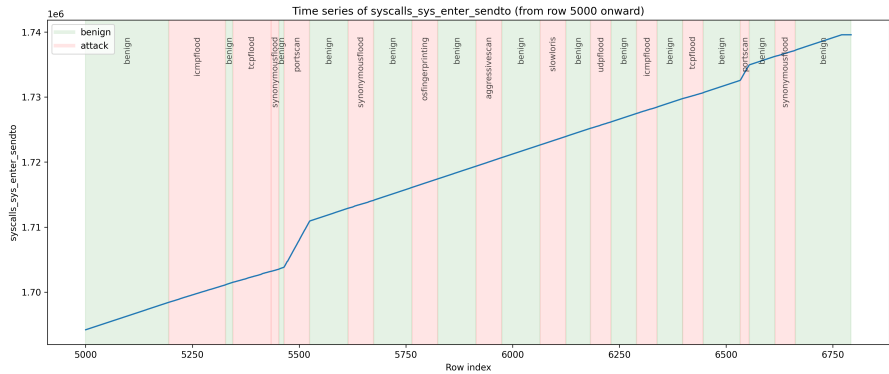


Figure 37: syscalls_sys_enter_sendto timeline

When we examine the key features, we can clearly observe that the highest peaks occur during flooding attacks, as previously identified. Interestingly, two of the key features also show increased activity during port scan attacks. This leads to the insight that these features could potentially be used to detect such attacks as well. When we now test our LSTM algorithm on the test dataset, we observe exactly this expected result.

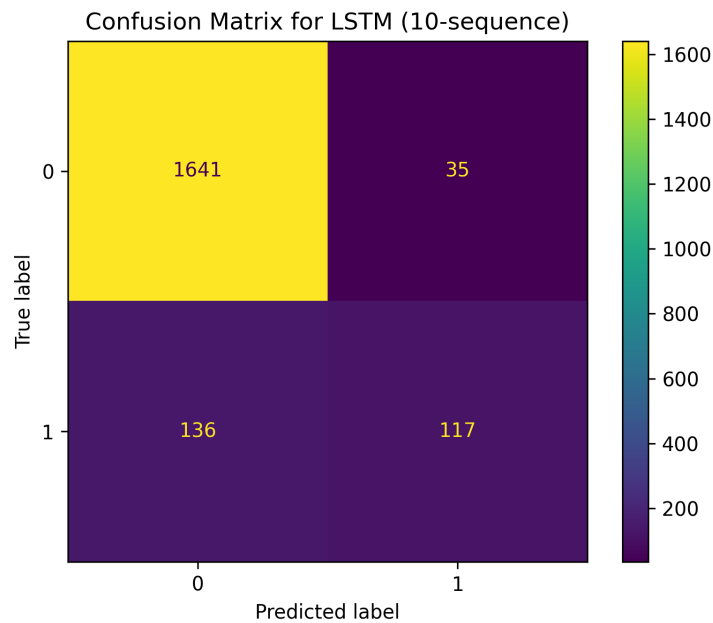


Figure 38: Confusion matrix

	misclassified	total
attack		
osfingerprinting	46	51
aggressivescan	39	51
slowloris	35	37
portscan	16	51
synonymousflood	0	60
tcpflood	0	3

Figure 39: Falsely labeled attacks

The algorithm has the most difficulty detecting the OS fingerprinting, aggressive scan, and Slowloris attacks. The port scan attack, however, was detected in most cases—35 out of 51. In the following we test the lstm algorithm in the live environment, too with similar results.

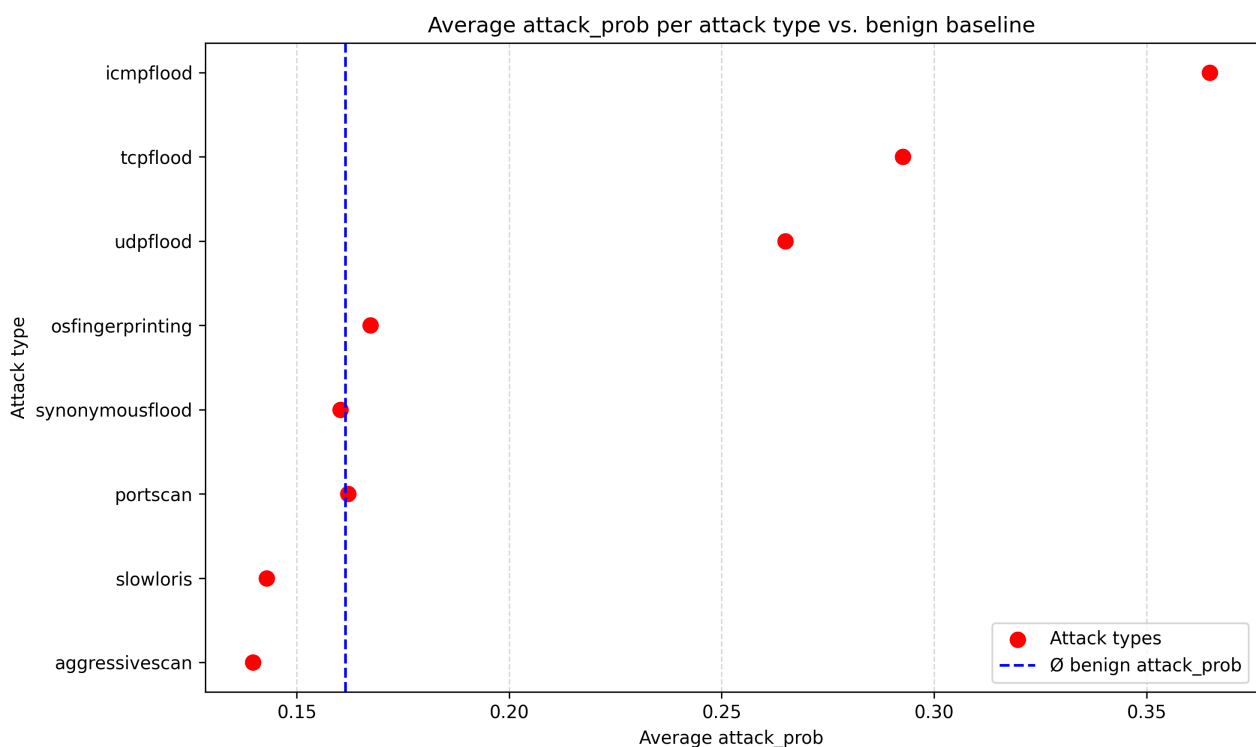


Figure 40: Average `attack_prob` values per attack type compared to the benign baseline. Flooding attacks show significantly higher average probabilities than the benign reference, while slow and scanning attacks remain close to or even below it.

Attack detection confidence analysis. Figure 40 shows the average `attack_prob` values for different attack types, as estimated by the classifier, in comparison to the average value observed during benign traffic. The vertical dashed line represents the benign baseline (approximately 0.1614).

Flooding attacks such as *ICMP flood*, *UDP flood*, and *TCP flood* exhibit clearly distinguishable `attack_prob` values, often well above 0.25, with *ICMP flood* peaking around 0.36. These high

values indicate strong model confidence and allow for reliable detection.

In contrast, attacks like *Slowloris*, *Aggressive Scan*, *Portscan*, and *OS Fingerprinting* present significantly lower `attack_prob` values, frequently falling below or very close to the benign average. This suggests that these attacks are more likely to be misclassified or overlooked, highlighting a potential weakness in the current detection model.

These results align with real-world test observations, where slower and stealthier attacks blend more seamlessly with normal traffic patterns, making them more challenging to detect. Future work should explore enhanced feature extraction or anomaly-based methods to improve classification confidence for these low-profile attacks.

9 Discussion

10 Conclusion/Future Work

einstellen das der pi immer eine gewissen bereich für die detection nutzen kann bei starken flooding angriffen, ist er nämlich so stark eingeschränkt das er nicht einmal mehr feedback der event daten geben kann.

Literaturverzeichnis

- [20125] Prometheus Authors 2014-2025. *Prometheus - Open source metrics and monitoring for your systems and services*. Zugriff am 22.07.2025. 2025. URL: <https://prometheus.io/>.
- [ACEA23] European Automobile Manufacturers Association (ACEA). *Interactive Map: Correlation Between Electric Car Sales and Charging Point Availability - 2023 Data*. <https://www.acea.auto/figure/interactive-map-correlation-between-electric-car-sales-and-charging-point-availability-2023-data/>. Accessed: 2025-07-09. 2023.
- [AI-Definition] Iqbal H. Sarker. "AI-Based Modeling: Techniques, Applications and Research Issues Towards Automation, Intelligent and Smart Systems". In: *SN Computer Science* 3.2 (2022). Accessed: 2025-07-10, p. 158. DOI: 10.1007/s42979-022-01043-x. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8830986/>.
- [Ali20] Manoj Basnet; Mohd. Hasan Ali. "Deep Learning-based Intrusion Detection System for Electric Vehicle Charging Station". In: (2020). Accessed: 2025-07-16. URL: <https://ieeexplore.ieee.org/document/9243152>.
- [Aro+21] Raman Arora et al. "Dropout: Explicit Forms and Capacity Control". In: (2021). Accessed: 2025-07-27. URL: <https://proceedings.mlr.press/v139/arora21a/arora21a.pdf>.
- [ASVIN] ASVIN. *ASVIN*. <https://asvin.io/>. Accessed: 2025-07-10. 2025.
- [BAN09] ARINDAM BANERJEE. "Anomaly Detection: A Survey". In: *University of Minnesota* (2009). Accessed: 2025-07-15. URL: <http://cucis.ece.northwestern.edu/projects/DMS/publications/AnomalyDetection.pdf>.
- [BMM24] Pragathi B.C., Hrithik Maddirala, and Sneha M. "Implementing an Effective Infrastructure Monitoring Solution with Prometheus and Grafana". In: (2024). Accessed: 2025-07-22. URL: <https://www.ijcaonline.org/archives/volume186/number38/pragathi-2024-ijca-923873.pdf>.
- [Bre01] Leo Breiman. "Random Forests". In: (2001). Accessed: 2025-07-15. URL: <https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>.
- [ChargeIQ] ChargeIQ GmbH. *ChargeIQ GmbH*. <https://chargeiq.de/>. Accessed: 2025-07-10. 2025.

- [coma] GeeksforGeeks community. *Types of Cache Misses*. URL: <https://www.geeksforgeeks.org/computer-organization-architecture/types-of-cache-misses/> (visited on 08/15/2025).
- [comb] The kernel development community. *Bus-Independent Device Accesses*. URL: <https://docs.kernel.org/next/driver-api/device-io.html> (visited on 08/17/2025).
- [comc] The kernel development community. *Event Tracing*. URL: <https://www.kernel.org/doc/html/v5.0/trace/events.html> (visited on 08/15/2025).
- [comd] The kernel development community. *What is NUMA?* URL: <https://www.kernel.org/doc/html/v5.6/vm/numa.html> (visited on 08/17/2025).
- [con25] hpc-wiki contributors. *Perf – Linux Performance Monitoring Tool*. <https://hpc-wiki.info/hpc/Perf>. Accessed: 2025-07-18. 2025.
- [Coo+17] Tim Cooijmans et al. “RECURRENT BATCH NORMALIZATION”. In: (2017). Accessed: 2025-07-27. URL: <https://arxiv.org/pdf/1603.09025>.
- [DecisionTreeExplain24] Bahzadour Taha Jijo and Adnan Mohsin Abdulazeez. “Decision Tree Explanation”. In: *Classification Based on Decision Tree Algorithm for Machine Learning* (2021). Accessed: 2025-07-09. URL: <https://jastt.org/index.php/jasttpath/article/view/65/24>.
- [Dro] Paul J. Drongowski. *PERF tutorial: Counting hardware performance events*. URL: <https://sandsoftwaresound.net/perf/perf-tut-count-hw-events/> (visited on 08/15/2025).
- [EDBF24] Sajjad Dadkhah Emmanuel Dana Buedi Ali A. Ghorbani and Raphael Lionel Ferreira. “Enhancing EV Charging Station Security Using a Multi-dimensional Dataset: CICEVSE2024”. In: (2024). Accessed: 2025-07-16. URL: https://link.springer.com/chapter/10.1007/978-3-031-65172-4_11.
- [EJP23] Raquel Espinosa, Fernando Jim´enez, and Jos´e Palm. “EMBEDDED FEATURE SELECTION IN LSTM NETWORKS WITH MULTI-OBJECTIVE EVOLUTIONARY ENSEMBLE LEARNING FOR TIME SERIES FORECASTING”. In: (2023). Accessed: 2025-08-17. URL: <https://arxiv.org/pdf/2312.17517>.

- [EntryPoints25] Md. Jakir Hossen V. Thiruppathy Kesavan. “Anomaly detection with grid sentinel framework for electric vehicle charging stations in a smart grid environment”. In: *Scientific Reports* 15.1 (2025). Accessed: 2025-07-09, p. 400. DOI: 10.1038/s41598-025-00400-z. URL: <https://www.nature.com/articles/s41598-025-00400-z>.
- [EU2050] European Commission. *2050 long-term strategy: Climate-neutral Europe by 2050*. https://climate.ec.europa.eu/eu-action/climate-strategies-targets/2050-long-term-strategy_en. Accessed: 2025-07-09. 2020.
- [EVSE-Cyber25] Paolo Bertoldi et al. “Cybersecurity for Electric Vehicle Charging Infrastructure: Challenges and Best Practices”. In: *World Electric Vehicle Journal* 15.12 (2025). Accessed: 2025-07-09, p. 571. DOI: 10.3390/wevj15120571. URL: <https://www.mdpi.com/2032-6653/15/12/571>.
- [EVSE24] Sagar Babu Mitikiri et al. “Regression Based Anomaly Detection in Electric Vehicle State of Charge Fluctuations Through Analysis of EVCI Data”. In: *arXiv preprint arXiv:2401.01580v1* (2024). Accessed: 2025-07-09. URL: <https://arxiv.org/abs/2401.01580v1>.
- [GDPR5] European Union. *General Data Protection Regulation (GDPR) – Article 5: Principles relating to processing of personal data*. <https://gdpr-info.eu/art-5-gdpr/>. Accessed: 2025-07-10. 2018.
- [GDPR6] European Union. *General Data Protection Regulation (GDPR) – Article 6: Lawfulness of processing*. <https://gdpr-info.eu/art-6-gdpr/>. Accessed: 2025-07-10. 2018.
- [Goo] *Exploratory Data Analysis for Feature Selection in Machine Learning*. https://services.google.com/fh/files/misc/exploratory_data_analysis_for_feature_selection_in_machine_learning.pdf. Accessed: 2025-07-23. Google LLC, 2024.
- [Gre] Brendan Gregg. *perf Examples*. URL: <https://www.brendangregg.com/perf.html> (visited on 08/15/2025).
- [HA04] V. J. Hodge and J. Austin. “A Survey of Outlier Detection Methodologies”. In: *Artificial Intelligence Review* (2004). Accessed: 2025-07-15. URL: https://www-users.york.ac.uk/~vjh5/myPapers/Hodge+Austin_OutlierDetection_AIRE381.pdf.
- [Han25] Yannic Hanel. *Interior view of a wallbox*. Own image. 2025.

- [Hod19] Daniel Hodges. *perf_exporter: A perf exporter for prometheus*. https://github.com/hodgesds/perf_exporter. Version v0.0.1, Zugriff am 18.07.2025. 2019.
- [IEA22] International Energy Agency. *Global EV Outlook 2022*. Accessed: 2025-07-09. International Energy Agency, 2022. URL: <https://www.iea.org/reports/global-ev-outlook-2022>.
- [KB15] Diederik P. Kingma and Jimmy Lei Ba. “ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION”. In: (2015). Accessed: 2025-07-27. URL: <https://arxiv.org/pdf/1412.6980>.
- [Kot24] Shahir Kottilingal. “COMPARATIVE ANALYSIS OF FEATURE SELECTION TECHNIQUES FOR LSTM BASED NETWORK INTRUSION DETECTION MODELS”. In: (2024). Accessed: 2025-07-26. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4979065.
- [Laux24] Lea Laux. “Übersicht eines Penetration Testings von Ladeinfrastruktur mit Schwerpunkt Ladestationen (im ReSiLENT-Projekt)”. Summer semester 2024, submission: August 2024. Bachelorarbeit. Ostbayerische Technische Hochschule Regensburg, 2024.
- [lim] arm limited. *PMU events*. URL: <https://developer.arm.com/documentation/100442/0100/debug-descriptions/pmu/pmu-events?lang=en> (visited on 08/15/2025).
- [Lim+14] Robert V. Lim et al. “Computationally Efficient Multiplexing of Events on Hardware Counters”. In: (2014). Accessed: 2025-07-18. URL: <https://www.kernel.org/doc/ols/2014/ols2014-lim.pdf>.
- [LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. “Isolation Forest”. In: (2008). Accessed: 2025-07-15. URL: <http://www.lamda.nju.edu.cn/publication/icdm08b.pdf>.
- [Mah22] Kiran Maharana. “A review: Data pre-processing and data augmentation techniques”. In: *ScienceDirect - Knowledge and Information Systems* (2022). Accessed: 2025-07-15. URL: <https://www.sciencedirect.com/science/article/pii/S2666285X22000565>.
- [Per19] PeruriVenkataAnusha. “Anomaly Detection Techniques”. In: *International Journal of Innovative Technology and Exploring Engineering* (2019). Accessed: 2025-07-15. URL: <https://www.ijitee.org/wp-content/uploads/papers/v9i1/A3910119119.pdf>.

- [Rah+25] Md Rayhanur Rahman et al. “Mining Temporal Attack Patterns from Cyberthreat Intelligence Reports”. In: (2025). Accessed: 2025-07-27. URL: <https://ar5iv.labs.arxiv.org/html/2401.01883>.
- [Ram] David Ramos. *Cybercriminals Target Electric Vehicle Chargers*. URL: <https://www.silicon.eu/cybercriminals-target-electric-vehicle-chargers-17047.html> (visited on 07/29/2025).
- [Ram+18] Daniel Ramos et al. “Deconstructing Cross-Entropy for Probabilistic Binary Classifiers”. In: (2018). Accessed: 2025-07-27. URL: <https://www.mdpi.com/1099-4300/20/3/208>.
- [ReSiLENT] ReSiLENT Project. *ReSiLENT Project*. <https://resilent-project.de/>. Accessed: 2025-07-10. 2025.
- [Sae] Mehreen Saeed. *A Gentle Introduction To Sigmoid Function*. URL: <https://machinelearningmastery.com/a-gentle-introduction-to-sigmoid-function/> (visited on 07/27/2025).
- [Skb] *The socket buffer*. URL: <http://oldvger.kernel.org/~davem/skb.html> (visited on 08/17/2025).
- [SM19] Ralf C. Staudemeyer and Eric Rothstein Morris. “Understanding LSTM – a tutorial into Long Short-Term Memory Recurrent Neural Networks”. In: (2019). Accessed: 2025-07-09. URL: <https://arxiv.org/pdf/1909.09586>.
- [SM24] Inna Stetsenko and Anton Myroniuk. “SOFTWARE FOR COLLECTING AND ANALYZING METRICS IN HIGHLY LOADED APPLICATIONS BASED ON THE PROMETHEUS MONITORING SYSTEM”. In: (2024). Accessed: 2025-07-22. URL: <https://ela.kpi.ua/server/api/core/bitstreams/8138e408-804b-45f6-9842-9e7b41e50dc0/content>.
- [StoehrMobility] Stöhr Mobility GmbH. *Stöhr Mobility GmbH*. <https://www.stoehr-mobility.de/>. Accessed: 2025-07-10. 2025.
- [The25] The pandas development team. *pandas documentation, version 2.3.1*. Accessed: 2025-07-23. 2025. URL: <https://pandas.pydata.org/docs/>.
- [Wea15] Vincent M. Weaver. “Linux perf event Features and Overhead”. In: (2015). Accessed: 2025-07-18, pp. 1–8.
- [Wt25] Michael Waskom and the Seaborn development team. *seaborn: statistical data visualization*. Accessed: 2025-07-23. 2025. URL: <https://seaborn.pydata.org/>.

Anhang

```
1 package main
2 import (
3     "fmt"
4     "io/ioutil"
5     "net/http"
6     "os/exec"
7     "strconv"
8     "strings"
9     "time"
10
11     "gopkg.in/yaml.v2"
12 )
13
14 // Config represents the structure of the YAML configuration file.
15 // It includes high-level performance counters (HCP), software events (SW
16 // ), and hardware events (HW).
17 type Config struct {
18     HCP []string `yaml:"hcp"`
19     SW  []string `yaml:"sw"`
20     HW  []string `yaml:"hw"`
21 }
22
23 // Global variable to store the most recent performance data.
24 var perfData string
25
26 // getPerfData runs 'perf stat' with the specified events for a short
27 // duration (0.1s)
28 // and returns the raw output as a string.
29 func getPerfData(events []string) (string, error) {
30     eventList := strings.Join(events, ",")
31     cmd := exec.Command("perf", "stat", "-e", eventList, "--", "sleep", "
32         0.1")
33     output, err := cmd.CombinedOutput()
34     if err != nil {
35         return "", err
36     }
37     return string(output), nil
38 }
39
40 // parsePerfData filters and extracts only the relevant lines from perf
41 // output,
42 // based on the provided list of events.
43 func parsePerfData(data string, events []string) string {
44     lines := strings.Split(data, "\n")
45     var result []string
46     for _, line := range lines {
```



```
43     for _, event := range events {
44         if strings.Contains(line, event) {
45             result = append(result, line)
46             break
47         }
48     }
49 }
50 return strings.Join(result, "\n")
51 }
52
53 // formatForPrometheus converts the filtered performance data into
54 // Prometheus-compatible metrics format.
55 func formatForPrometheus(data string) string {
56     lines := strings.Split(data, "\n")
57     var result []string
58     for _, line := range lines {
59         parts := strings.Fields(line)
60         if len(parts) >= 2 {
61             // Clean up the metric name
62             metricName := strings.ReplaceAll(parts[1], ".", "_")
63             // metricName = strings.ReplaceAll(metricName, "-", "_")
64             // Remove decimal points from the number and parse it
65             metricValue := strings.ReplaceAll(parts[0], ".", "")
66             if value, err := strconv.ParseFloat(metricValue, 64); err == nil
67             {
68                 // Format value appropriately for Prometheus (integer or
69                 // float)
70                 if value == float64(int(value)) {
71                     result = append(result, fmt.Sprintf("%s %d", metricName,
72                     int(value)))
73                 } else {
74                     result = append(result, fmt.Sprintf("%s %f", metricName,
75                     value))
76                 }
77             }
78         }
79     }
80     return strings.Join(result, "\n")
81 }
82
83 // metricsHandler handles HTTP requests and returns the formatted
84 // performance metrics.
85 func metricsHandler(w http.ResponseWriter, r *http.Request) {
86     fmt.Fprintf(w, formatForPrometheus(perfData))
87 }
88
89 // chunkEvents splits the list of events into smaller chunks of a given
90 // size.
```

```
84 // This helps avoid issues with perf's limit on the number of
    // simultaneous counters.
85 func chunkEvents(events []string, chunkSize int) [][]string {
86     var chunks [][]string
87     for i := 0; i < len(events); i += chunkSize {
88         end := i + chunkSize
89         if end > len(events) {
90             end = len(events)
91         }
92         chunks = append(chunks, events[i:end])
93     }
94     return chunks
95 }
96
97 // validateEvent checks if a given performance event is available on the
    // system by using 'perf list'.
98 func validateEvent(event string) bool {
99     cmd := exec.Command("perf", "list")
100     output, err := cmd.CombinedOutput()
101     if err != nil {
102         fmt.Println("Error listing perf events:", err)
103         return false
104     }
105     return strings.Contains(string(output), event)
106 }
107
108 func main() {
109     // Load the YAML configuration file
110     config := Config{}
111     data, err := ioutil.ReadFile("config.yml")
112     if err != nil {
113         fmt.Println("Error reading config file:", err)
114         return
115     }
116     err = yaml.Unmarshal(data, &config)
117     if err != nil {
118         fmt.Println("Error parsing config file:", err)
119         return
120     }
121
122     // Validate and collect all available events from the config
123     var validEvents []string
124     for _, event := range append(config.HCP, config.SW...) {
125         if validateEvent(event) {
126             validEvents = append(validEvents, event)
127         }
128     }
129     for _, event := range config.HW {
```

```
130     if validateEvent(event) {
131         validEvents = append(validEvents, event)
132     }
133 }
134
135 // Split the events into manageable chunks (TODO: dynamically adjust
136 // based on CPU performance)
137 eventChunks := chunkEvents(validEvents, 6)
138
139 // Start a background goroutine to periodically collect performance
140 // data
141 go func() {
142     for {
143         var allPerfData []string
144         for _, chunk := range eventChunks {
145             data, err := getPerfData(chunk)
146             if err != nil {
147                 fmt.Println("Error:", err)
148                 allPerfData = append(allPerfData, "Error fetching data")
149             } else {
150                 allPerfData = append(allPerfData, parsePerfData(data,
151                     chunk))
152             }
153         }
154         // Combine data from all chunks into one string for Prometheus
155         // export
156         perfData = strings.Join(allPerfData, "\n")
157         time.Sleep(100 * time.Millisecond)
158     }
159 }()
160
161 // Start the HTTP server that exposes the '/metrics' endpoint for
162 // Prometheus scraping
163 http.HandleFunc("/metrics", metricsHandler)
164 fmt.Println("Starting server on :9011")
165 if err := http.ListenAndServe(":9011", nil); err != nil {
166     fmt.Println("Error starting server:", err)
167 }
168 }
```

Listing 9: Go script for performance monitoring via perf for Prometheus

```
1 import requests
2 import csv
3 from datetime import datetime, timedelta
4 from collections import defaultdict
5 import numpy as np
6
7 from config_allevents import FEATURES
```

```
8
9 PROMETHEUS_URL = "http://192.168.1.25:9090"
10 STEP = "10" # seconds
11 CHUNK_HOURS = 2
12
13 start_time = datetime.fromisoformat("2025-07-22T18:00:00.000000")
14 end_time = datetime.fromisoformat("2025-07-23T14:40:00.000000")
15
16 def build_chunks(start, end, chunk_hours):
17     chunks = []
18     chunk_delta = timedelta(hours=chunk_hours)
19     current_start = start
20     while current_start < end:
21         current_end = min(current_start + chunk_delta, end)
22         chunks.append((
23             current_start.isoformat("T") + "Z",
24             current_end.isoformat("T") + "Z"
25         ))
26         current_start = current_end
27     return chunks
28
29 def query_metric_chunked(metric, chunks):
30     all_results = []
31     for chunk_start, chunk_end in chunks:
32         url = f"{PROMETHEUS_URL}/api/v1/query_range"
33         params = {
34             "query": metric,
35             "start": chunk_start,
36             "end": chunk_end,
37             "step": STEP,
38         }
39         response = requests.get(url, params=params)
40         response.raise_for_status()
41         all_results.extend(response.json()["data"]["result"])
42     return all_results
43
44 def main():
45     chunks = build_chunks(start_time, end_time, CHUNK_HOURS)
46     data = defaultdict(dict)
47     timestamps = set()
48     values_dict = {}
49     metric_query_map = {}
50
51     for feat in FEATURES:
52         prom_name = feat.replace("-", "_")
53         url = f"{PROMETHEUS_URL}/api/v1/query"
54         hit = False
55
```

```
56     # 1. sum(prom_name) abfragen
57     query = f"sum({prom_name})"
58     params = {'query': query}
59     try:
60         resp = requests.get(url, params=params)
61         resp.raise_for_status()
62         results = resp.json()['data']['result']
63         if results:
64             value = float(results[0]['value'][1])
65             values_dict[feat] = value
66             metric_query_map[feat] = query
67             hit = True
68     except Exception:
69         pass
70
71     if hit:
72         continue
73
74     # 2. sum(perf_prom_name) abfragen
75     prom_name_perf = 'perf_' + prom_name
76     query = f"sum({prom_name_perf})"
77     params = {'query': query}
78     try:
79         resp = requests.get(url, params=params)
80         resp.raise_for_status()
81         results = resp.json()['data']['result']
82         if results:
83             value = float(results[0]['value'][1])
84             values_dict[feat] = value
85             metric_query_map[feat] = query
86             hit = True
87     except Exception:
88         pass
89
90     values_array = []
91     feature_names_for_array = []
92
93     for feat in FEATURES:
94         feature_names_for_array.append(feat)
95         values_array.append(values_dict.get(feat, np.nan))
96
97     for feat, querystr in metric_query_map.items():
98         results = query_metric_chunked(querystr, chunks)
99         for series in results:
100             for timestamp, value in series["values"]:
101                 ts = datetime.utcfromtimestamp(float(timestamp)).
102                     isoformat()
103                 data[ts][feat] = value
```

```

103         timestamps.add(ts)
104
105     sorted_timestamps = sorted(timestamps)
106
107     all_columns = list(FEATURES)
108
109     with open("evse_metrics_wide.csv", "w", newline="") as f:
110         writer = csv.writer(f)
111         header = ["timestamp"] + all_columns
112         writer.writerow(header)
113         for ts in sorted_timestamps:
114             row = [ts] + [data[ts].get(col, "") for col in all_columns]
115             writer.writerow(row)
116
117 if __name__ == "__main__":
118     main()

```

Listing 10: Python script for scraping performance monitoring from Prometheus into an csv file

```

1  import subprocess
2  import time
3  from datetime import datetime
4
5  commands = [
6      ("Slowloris", ["python3", "slowloris.py", "192.168.1.25", "-s", "500"
7          ]),
8      ("UDP Flood", ["sudo", "hping3", "--flood", "--udp", "-p", "80", "
9          192.168.1.25"]),
10     ("ICMP Flood", ["sudo", "hping3", "--flood", "-1", "192.168.1.25"]),
11     ("TCP Flood", ["sudo", "hping3", "--flood", "-p", "80", "192.168.1.25"
12         ]),
13     ("Synonymous Flood", ["sudo", "hping3", "--flood", "--rand-source", "-p", "80", "192.168.1.25"]),
14     ("Portscan", ["bash", "-c", "for i in {1..1000}; do sudo nmap -sT
15         192.168.1.25; sleep 1; done"]),
16     ("OS Fingerprinting", ["bash", "-c", "for i in {1..1000}; do sudo
17         nmap -O 192.168.1.25; sleep 1; done"]),
18     ("Aggressive Scan", ["bash", "-c", "for i in {1..1000}; do sudo nmap
19         -A 192.168.1.25; sleep 1; done"]),
20 ]
21
22 LOG_FILE = "timestamp_attacks.txt"
23 RUN_TIME = 15 * 60      # 15 Minuten
24 BREAK_TIME = 20 * 60   # 20 Minuten
25
26 def log_msg(msg):
27     with open(LOG_FILE, "a") as f:
28         f.write(msg + "\n")

```

```

23
24 while True:
25     for name, cmd in commands:
26         start_time = datetime.now()
27         print(f"Starte {name}: {' '.join(cmd)}")
28         log_msg(f"Start: {start_time.strftime('%Y-%m-%d %H:%M:%S')} -
                Angriff: {name} - Befehl: {' '.join(cmd)}")
29         try:
30             proc = subprocess.Popen(cmd)
31             proc.wait(timeout=RUN_TIME)
32             note = "Erfolgreich beendet"
33         except subprocess.TimeoutExpired:
34             proc.terminate()
35             try:
36                 proc.wait(10)
37             except Exception:
38                 proc.kill()
39             note = "Nach 15 Minuten abgebrochen"
40         end_time = datetime.now()
41         log_msg(f"Ende: {end_time.strftime('%Y-%m-%d %H:%M:%S')} ({note})
                ")
42         log_msg("Pause startet jetzt.\n")
43         time.sleep(BREAK_TIME)

```

Listing 11: Python script for automatet attacks and storage of the timestamps

content des beigefügten Datenträgers:

- ...
- ...

A Domändenmodell

Ein toller Anhang, der nicht nur als „*Müllhalde*“ genutzt wird, sondern in dem Bilder und contente auch mit eigenen Worten erklärt werden und den man auch für sich alleine lesen kann. Es sollten auch Referenzen auf die zugehörige ausführliche Behandlung im Hauptteil inklusive Seitenangabe mit \pageref gegeben werden.

Screenshot

Unterkategorie, die nicht im contentsverzeichnis auftaucht.